

Inżynieria Internetu Rzeczy

Oprogramowanie inżynierskie do pracy grupowej

Materiały do warsztatów nr 4

Kierunek studiów: Inżynieria Internetu Rzeczy i Telekomunikacja
Przedmiot: PBL
Kod przedmiotu: 103A-IRxxx-ISP-PBL

dr inż. Mrozek Tomasz
tomasz.mrozek@pw.edu.pl

Zakres wiedzy

W ramach laboratorium/warsztatu nauczysz się:

- podstaw obsługi systemu kontroli wersji Git;
- tworzenia repozytoriów lokalnych i zdalnych;
- śledzenia zmian w plikach i zarządzania historią projektu;
- wykonywania operacji add, commit, push i pull;
- tworzenia i przełączania się między gałęziami (branch);
- łączenia zmian z różnych gałęzi (merge);
- rozwiązywania konfliktów podczas łączenia zmian;
- współpracy zespołowej z wykorzystaniem platformy GitHub;
- stosowania dobrych praktyk w zarządzaniu kodem źródłowym.

Wymagania do zaliczenia laboratorium/warsztatu:

W celu zaliczenia laboratorium należy:

- mieć obecność na zajęciach,
- wykonać polecenia zgodnie z instrukcją,
- wykonać ewentualne dodatkowe polecenia zadane przez prowadzącego na zajęciach.

Ocena z laboratorium/warsztatu:

Informacje dotyczące oceny z laboratorium/warsztatu:

- w ramach laboratorium/warsztatu można uzyskać maksymalnie 2 punkty.
- 2 punkty można uzyskać za poprawne wykonanie instrukcji potwierdzone przedstawieniem uzyskanych rezultatów prowadzącemu wraz z ewentualnym wykonaniem poleceń zadanych przez prowadzącego.

Uwagi do instrukcji:

- należy dokładnie czytać instrukcje,
- jeśli wystąpi jakiś błąd lub coś nie działa, jak powinno, należy się upewnić, czy wszystko zostało wykonane zgodnie z instrukcją.

Uwaga: w przedstawionej instrukcji w omawianych przykładach mogą występować foldery lub pliki. Należy je samodzielnie stworzyć.

Część obrazków demonstrujących operacje na repozytorium z poziomu przeglądarki internetowej przedstawia starą wersję GIT (przed odświeżeniem szaty graficznej i dodaniu nowych opcji). Nie ma to jednak wpływu na powodzenie w wykonaniu przedstawionych przykładów.

1. Krótki wstęp do GIT

Oprogramowanie inżynierskie do pracy grupowej:

- nadzorowanie i kontrola postępów pracy,
- lepszy podział obowiązków i zadań,
- możliwość śledzenia zmian i dokonań wszystkich członków zespołu,
- możliwość wracania do poprzednich wersji przed zmianami.

GIT – system kontroli wersji

Definicja: System kontroli wersji (ang. version/revision control system) – oprogramowanie służące do śledzenia zmian głównie w kodzie źródłowym oraz pomocy programistom w łączeniu zmian dokonanych w plikach przez wiele osób w różnym czasie.

Zalety:
- doskonały do pracy przy projektach zespołowych,
- dobrze się sprawdza przy pracy samodzielnej

Po co:
- Chcesz wrócić do wersji kodu sprzed 2 tygodni, czwartek o 17:12. Albo do wczorajszej, bo wczoraj jeszcze działało, a jesteś absolutnie pewien, że nic nie zepsułeś.
- Nie pamiętasz, po co modyfikowałeś dany plik albo nawet konkretną linię.
- Fragment programu nie działa, nie wiesz od kiedy.
- Chcesz wykonać mały eksperyment „na boku", bez obaw, że zepsujesz „właściwą" wersję kodu.
- Chcesz móc sprawdzić, jak wyglądały Twoje postępy w projekcie.
- Chcesz łatwo opublikować swój kod w serwisie GitHub, aby się nim pochwalić lub pozwolić skorzystać komuś innemu.

Praca z gitem, repozytorium:

Praca z Gitem opiera się na używaniu repozytorium. Można powiedzieć, że jest to taka „baza danych" przechowująca informacje o aktualnym oraz historycznym stanie projektu. Na potrzeby początkujących można założyć, że jeden projekt to jedno repozytorium.

Repozytorium może być lokalne lub zdalne. Lokalne to takie, które posiadasz na swoim komputerze. Natomiast zdalne znajduje się w innej lokalizacji (np. na popularnym GitHubie), do której istnieje odwołanie w lokalnym repozytorium. Korzystanie z takiego modelu pozwala na to, by nad projektem jednocześnie pracowało wiele osób.

Commit:

Podstawową jednostką pracy jest commit, czyli zapisany stan repozytorium w danym momencie – trochę jak spakowany katalog projektu, tyle że dużo bardziej inteligentny. Zdecydowana większość codziennej pracy z Gitem opiera się na tworzeniu, przeglądaniu i modyfikowaniu commitów.

Każdy commit składa się z kilku elementów, m.in.:

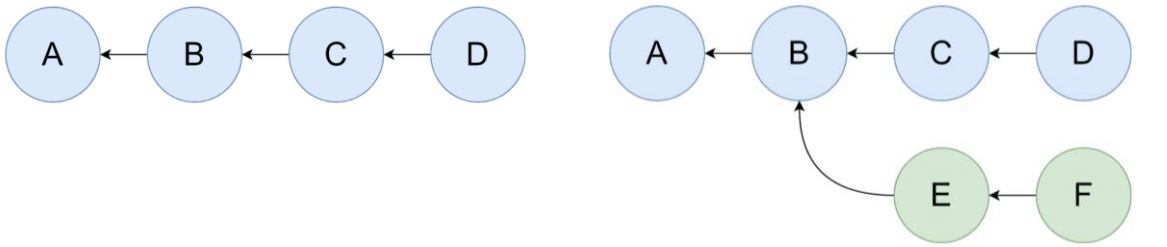
- zmian, które wprowadza,
- autora,

- daty utworzenia,
- komentarza,
- wartości hash (SHA-1),
- listy rodziców.

Commity są unikalne i rozróżnialne na podstawie hasha SHA-1. Jest on dosyć długi – w praktyce używa się jego skróconej wersji (pierwsze 6–8 znaków).

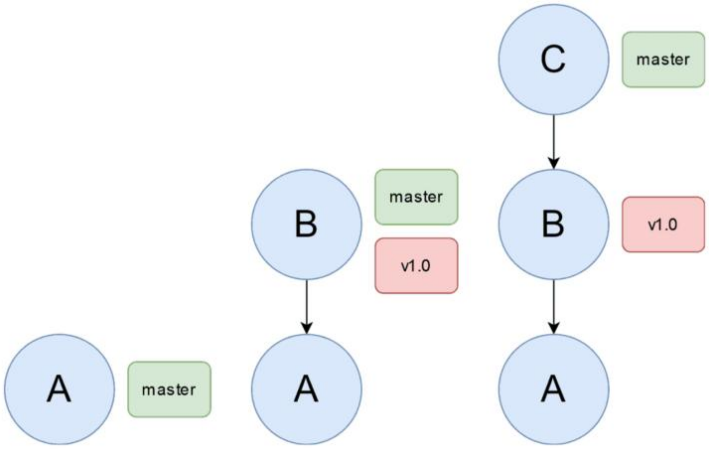
Drzewo:

Każdy commit (poza pierwszym) **ma przynajmniej jednego rodzica**, czyli commita, na bazie którego powstał. Tworzy to strukturę podobną do drzewa. Poniższy rysunek przedstawia dwa przykładowe drzewa. Lewe jest liniowe, natomiast prawe posiada rozgałęzienie. W obu przypadkach commit A jest pierwszy (jako jedyny nie posiada rodzica). Strzałka od X do Y oznacza „X jest potomkiem Y”.



Warto zapamiętać, że w każdym repozytorium zawsze istnieje domyślna gałąź o nazwie **master** (w przypadku niektórych repozytorium np. GitHub od 2020 roku nazwa „master” została zastąpiona przez „main”).

W przypadku liniowego projektu będzie to jedyna istniejąca gałąź. Oznacza to, że ostatnio utworzony commit (nowo utworzony) zawsze będzie miał dodatkową etykietę „master”, co zostało przedstawione na poniższym rysunku.



2. Narzędzia do pracy z GIT

Do pracy z GIT potrzebne jest konto w serwisie GitHub oraz asystent (program) do obsługi GIT na wykorzystywanym komputerze. Etap założenia konta oraz instalacji programu zostały przedstawione w części a) i b).

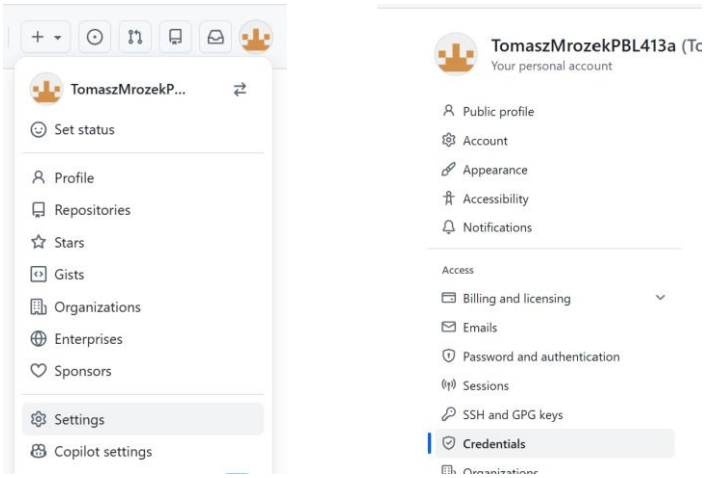
a) Konto w serwisie GitHub

Do wykonania laboratorium konieczne jest założenie konta w serwisie GitHub. Można to zrobić pod adresem:

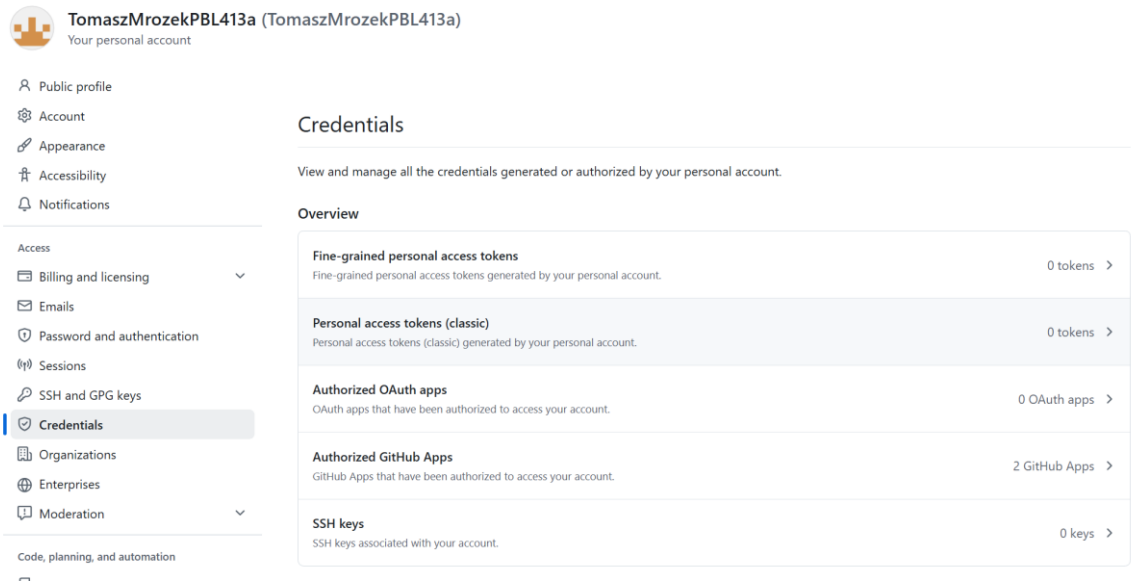
https://github.com/

Po założeniu konta warto przejść do ustawień **Settings -> Credentials**, aby nadać klucz (token). Będzie on niezbędny do zalogowania się do GitHub'a z poziomu konsoli.

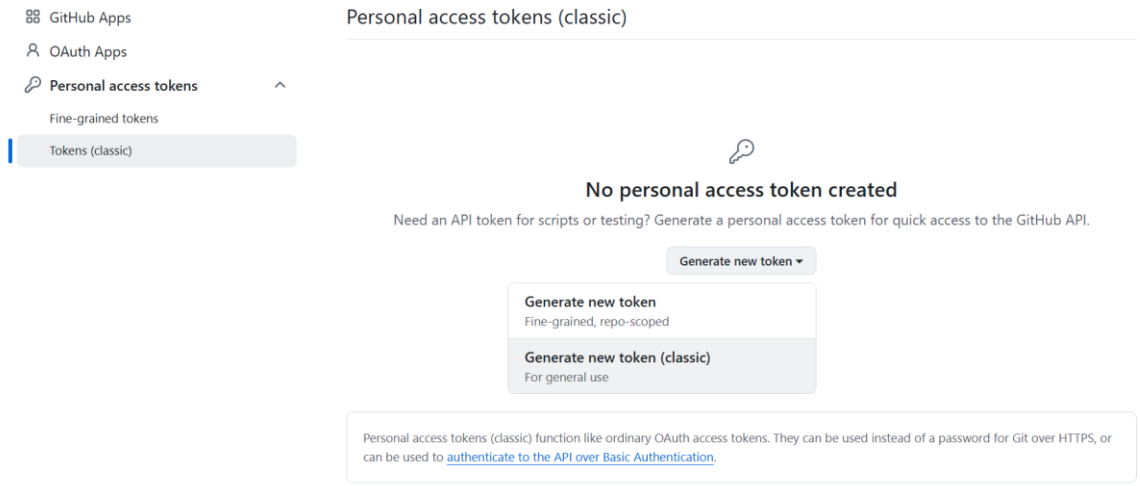
Po wejściu na stronę GitHub w przeglądarce internetowej należy się zalogować, a następnie kliknąć w prawym górnym rogu strony w awatar konta. Z listy rozwijanej należy wybrać opcję „Settings", a na nowej stronie z prawej strony w pasku z opcjami wybrać opcję „Credentials".



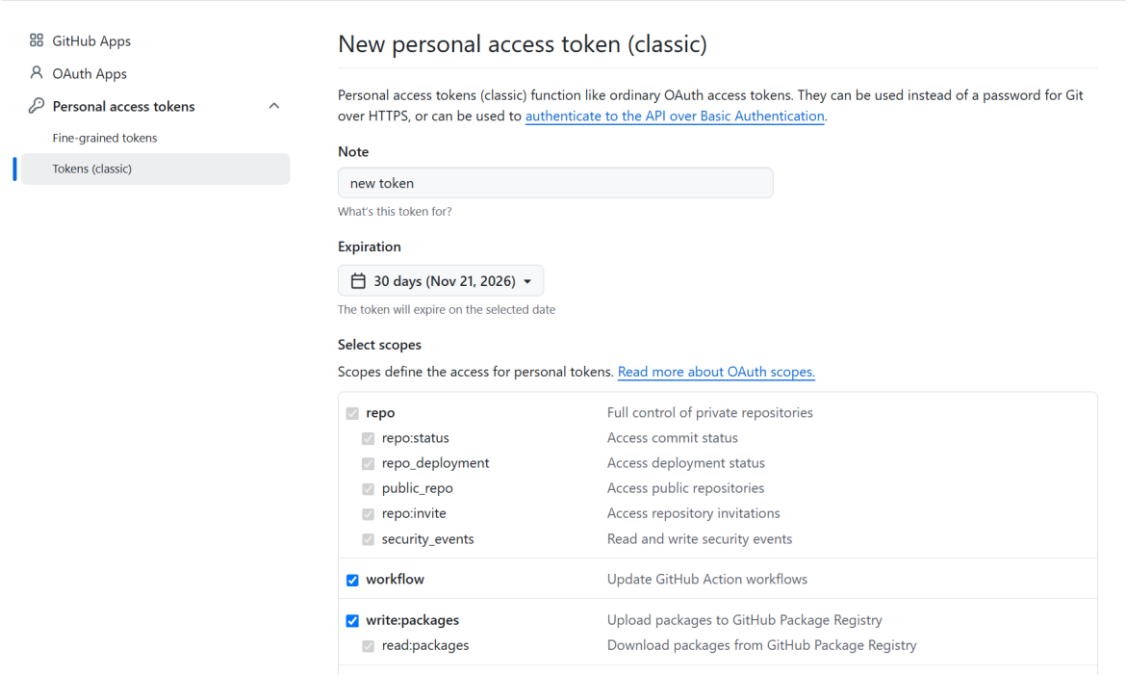
Następnie należy przejść do opcji „Personal access tokens (classic)".



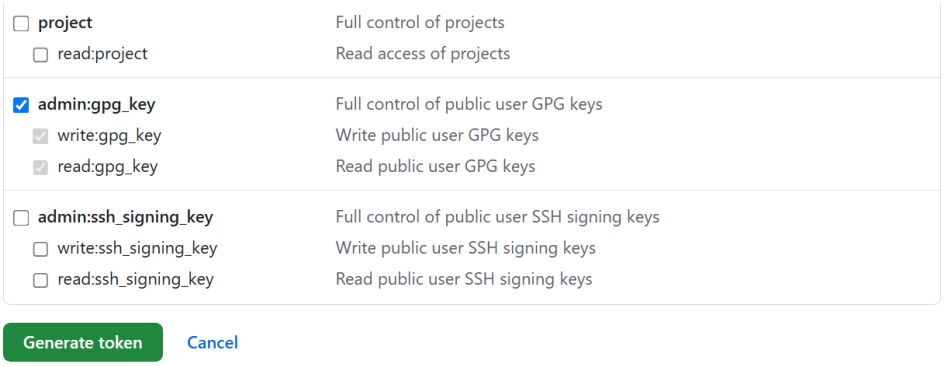
Na nowo otwartej stronie należy wybrać opcję „Generate new token” i wybrać tę z dopiskiem „Classic”.



Należy nadać nazwę tokena w polu „Note”, a następnie zaznaczyć wybrane poniższe opcje (workflow, write:package i admin:gpg_key). Można zaznaczyć wszystkie opcje. Należy jednak pamiętać, że po zaznaczeniu wszystkich opcji zyskuje się nieograniczony dostęp do wszystkich funkcjonalności, dlatego należy zadbać o odpowiednie zabezpieczenie klucza.

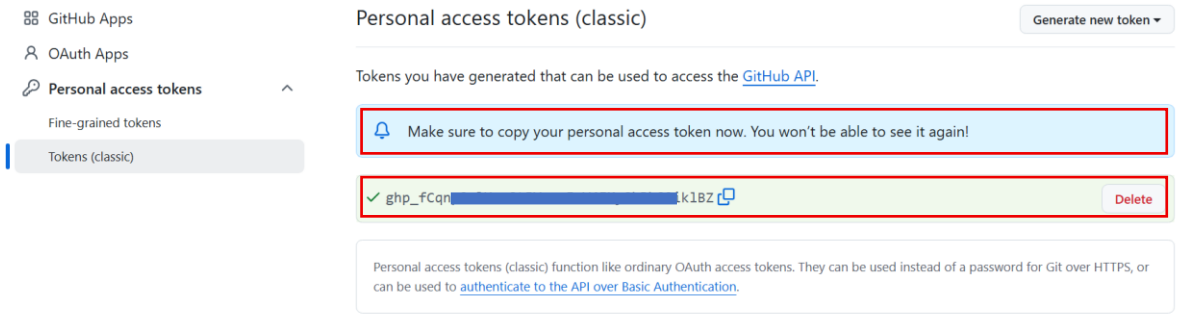


Ostatnim etapem po zaznaczeniu pożądanych opcji jest wybranie przycisku „Generate token” na samym dole strony.



Zostanie wyświetlona nowa strona z nowym kluczem widocznym w drugiej czerwonej ramce. Ten klucz można skopiować do pamięci podręcznej klikając w ikonę z zielonego kwadratu. Klucz taki można sobie np. skopiować do pliku tekstowego.

UWAGA (bardzo ważne!): Zgodnie z tekstem z pierwszej poniższej ramki wygenerowany klucz zostanie wyświetlony tylko raz. Z tego względu zalecane jest, aby go gdzieś zapisać np. w pliku tekstowym, gdyż klucz ten będzie wymagany do logowania. Jeśli tego nie zrobimy konieczne będzie generowanie nowego klucza.



b) Program Git dla używanego systemu operacyjnego

Po założeniu konta w GitHub należy pobrać odpowiedni program na komputer. Preferowany program do obsługi GIT-a dla najpopularniejszych systemów operacyjnych (Windows, Linux, iOS) jest dostępny w:

https://git-scm.com/downloads.

Opcje do wyboru w trakcie instalacji można pozostawić domyślne, na końcu można wybrać edytor tekstu: Notepad++.

Po instalacji Git będzie dostępny z każdego okna konsoli tj.:

- Wiersz polecenia,
- Windows PowerShell,
- Git Bash.

Dla systemu Windows, **Wiersz poleceń** oraz **Windows PowerShell** można uruchomić z pasku „Start” (klikając znaczek Windows – lewy dolny róg na ekranie), wpisując po jego kliknięciu nazwę szukanej konsoli.

Aby uruchomić **Git Bash** należy przejść do wybranego folderu, następnie kliknąć prawy przycisk myszy i z rozwijanej listy wybrać opcję „**Git Bash Here**”.

<u>Podstawowe polecenia</u> do poruszania się w konsolach:

ls	– wyświetlenie wszystkich dostępnych plików i folderów w danej lokalizacji
cd nazwa_folderu	– przejście do wskazanego folderu (np. cd Documents)
cd .. lub cd..	– powrót do katalogu nadrzędnego (dla Git Bash tylko cd ..)
clear, cls	– wyczyszczenie konsoli (dla Git Bash tylko clear, dla Wiersz polecenia tylko cls)

WAŻNE:

Przed rozpoczęciem korzystania z GIT-a należy przeprowadzić małą konfigurację. Należy przejść do preferowanej konsoli (Wiersz poleceń, Git Bash ..) i wpisać oddzielnie każdą z poniższych komend:

```
git config --global user.name "NAZWA UŻYTKOWNIKA"
git config --global user.email "adres@email"
```

gdzie: „NAZWA UŻYTKOWNIKA” i „adres@email” muszą być takie same jakie przyjęto do utworzenia konta na GitHubie.

np. (dla mojego konta utworzonego na GitHub):

```
git config --global user.name "TomaszMrozekPBL"
git config --global user.email "t.mrozek@tele.pw.edu.pl"
```

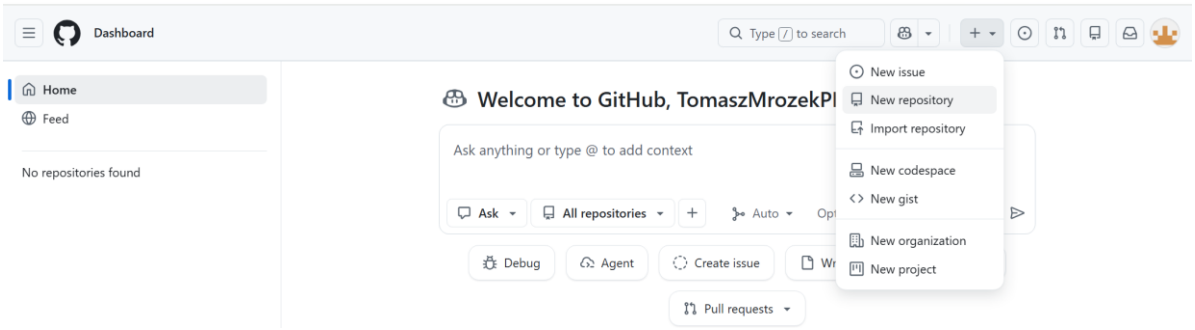
Wprowadzone dane będą widoczne przy każdym commit. Parametr –global oznacza, że opcja będzie ustawiana dla wszystkich repozytoriów.

3. Praca z GIT

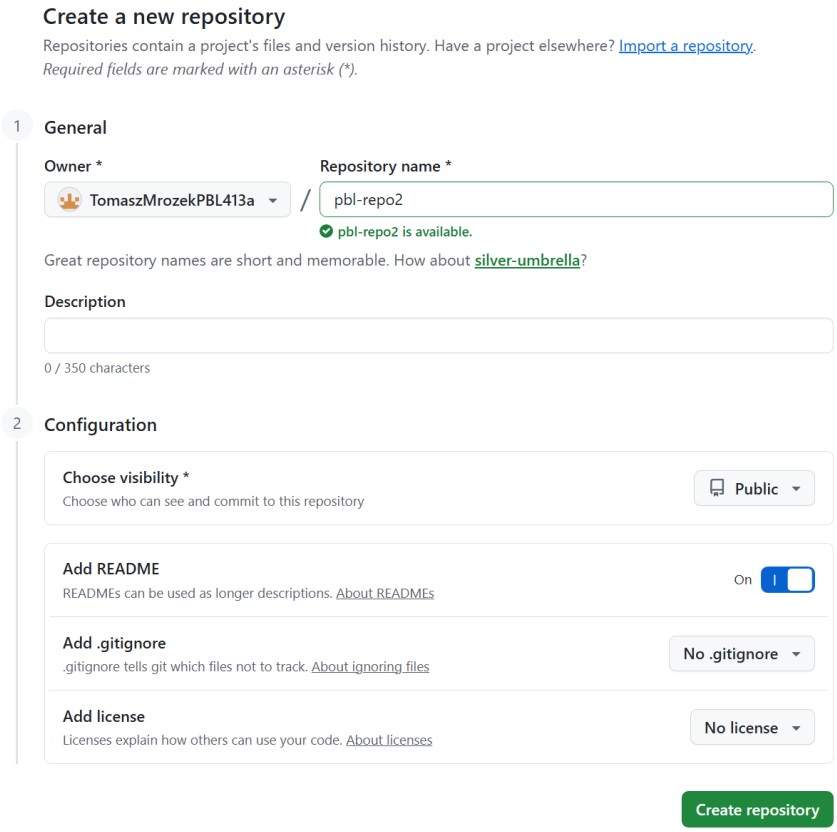
W kolejnych pod punktach przedstawione zostały etapy nauki z GIT. Czytaj uważnie wszystkie polecenia/objaśnienia. Nie skupiaj się tylko na załączonych obrazkach. Zanim wykonasz jakieś polecenie upewnij się, że jest one tożsame z obrazkiem (jeśli występuje). Pamiętaj, że pośpiech nie jest wskazany, a *diabeł tkwi w szczegółach*.

a) Pierwszy projekt na GitHubie

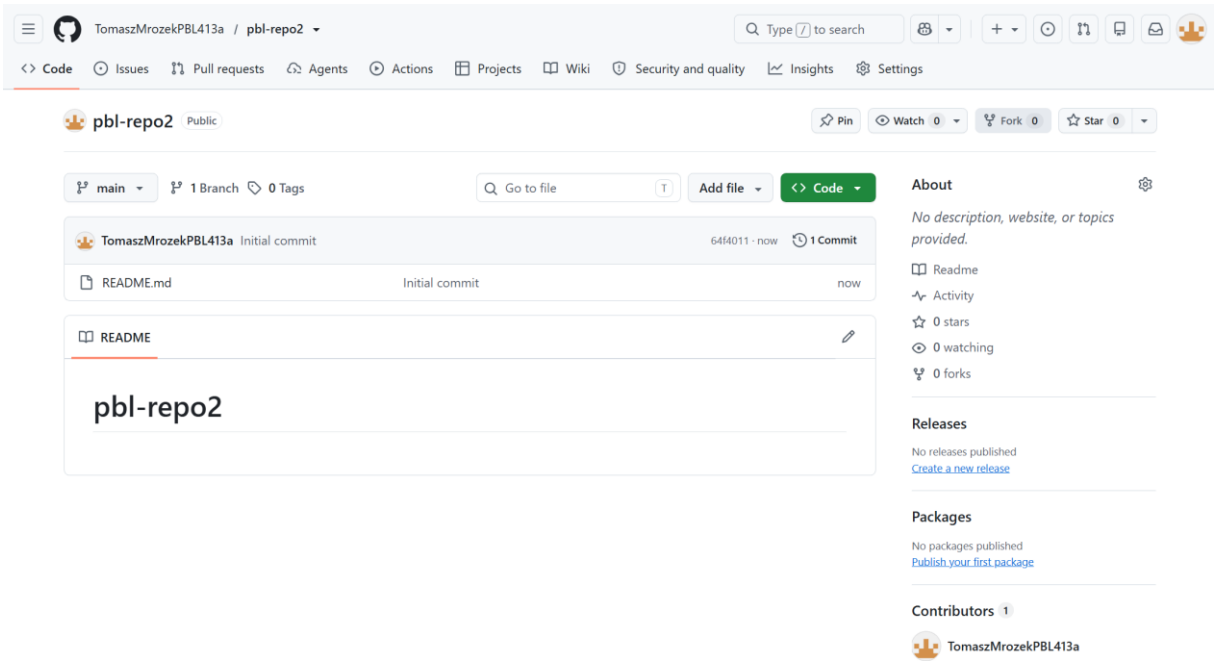
Aby rozpocząć pracę z GIT trzeba założyć nowe repozytorium. Należy zalogować się na swoje konto na stronie GitHub i postąpić zgodnie z poniższym rysunkiem. Po zalogowaniu kliknąć w znaczek +, a z rozwijanej listy wybrać opcję „New repository”.



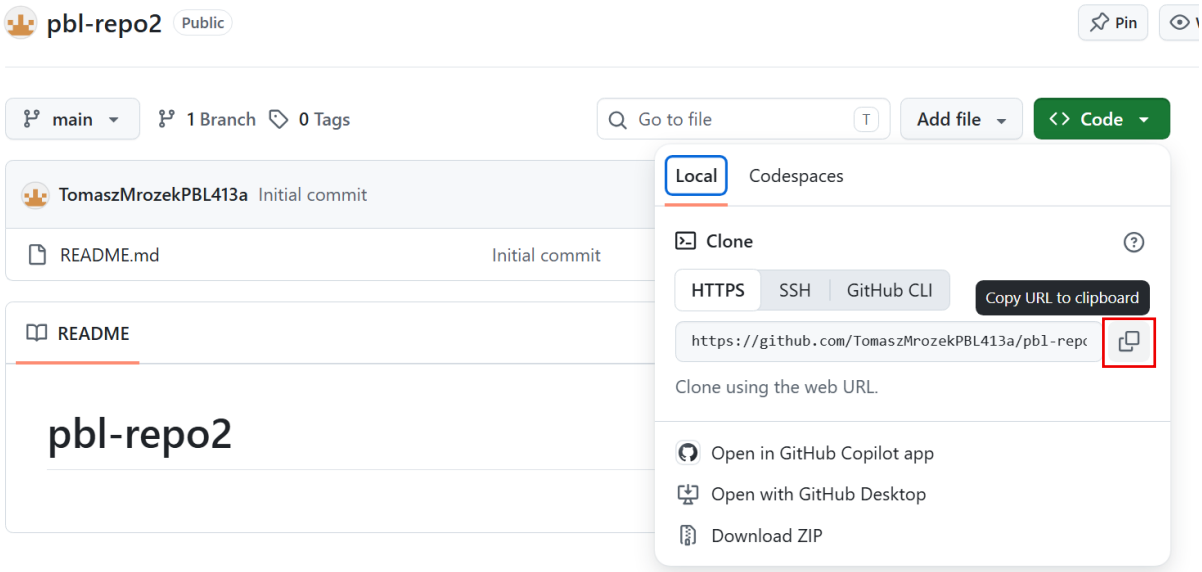
W nowo otwartym oknie należy nadać nazwę dla utworzonego repozytorium (tutaj pbl-repo2), wybrać widoczność repozytorium (tutaj „Public”, dla „Private” konieczne będzie wprowadzanie hasła dostępu z poziomu GIT’a) oraz dodać plik README (Add README) do projektu. Na samym końcu wybrać przycisk **Create repository**. W ten sposób zostanie utworzone i wyświetlone **repozytorium zdalne**.



Po utworzeniu repozytorium nastąpi przekierowanie do poniższej strony.



Kolejnym etapem jest utworzenie repozytorium lokalnego. Na nowo wyświetlonej stronie po utworzeniu repozytorium zdalnego, należy wybrać przycisk z listą rozwijaną **Code**, a następnie skopiować adres URL z zakładki HTTPS stworzonego repozytorium (klikając w ikonę z prawej strony linku https).

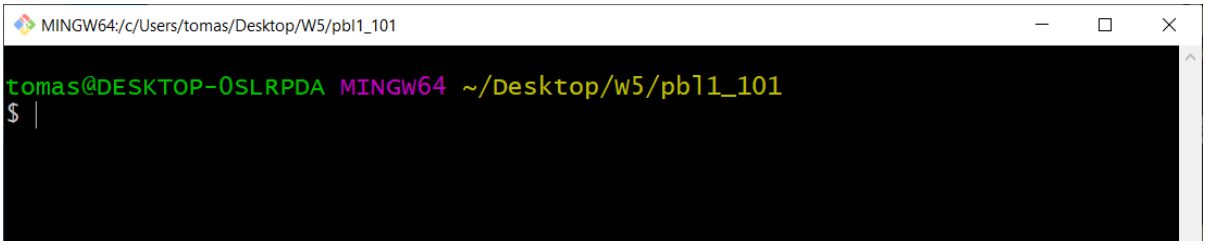


Po poprawnym skopiowaniu linku zostanie wyświetlony komunikat-dymek „Copied!”. W kolejnym etapie zostanie utworzone repozytorium lokalne z wykorzystaniem skopiowanego adresu URL.

b) Klonowanie repozytorium

Wykorzystując skopiowany wcześniej adres URL należy przejść do wybranego folderu, do którego chcemy skopiować repozytorium. Następnie należy uruchomić konsolę (np. klikając

prawym klawiszem myszy w wolną przestrzeń i wybierając opcję „Git Bash Here") w preferowanym folderze (np. pbl1_101). Uwaga, folder należy wcześniej ręcznie utworzyć.



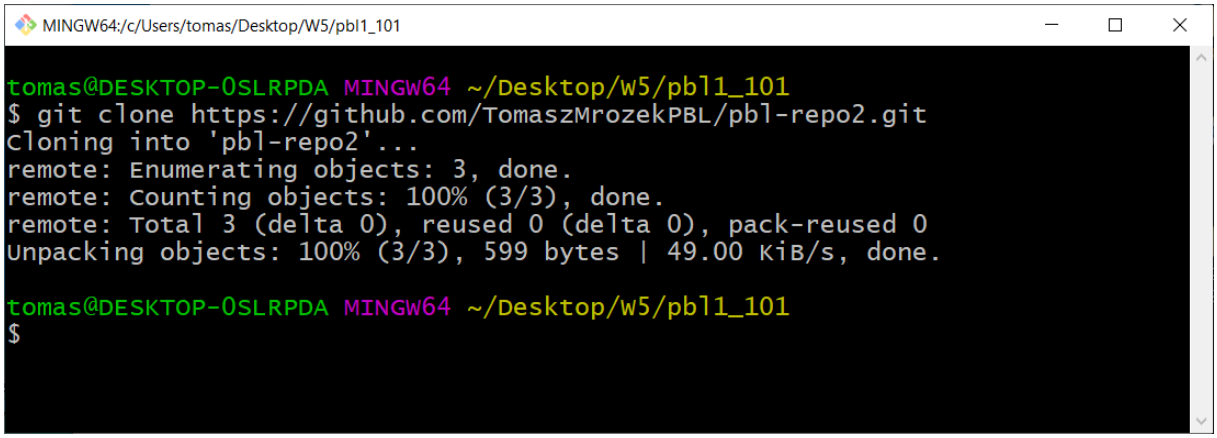
Następnie w celu skopiowania repozytorium należy wpisać w konsoli poniższą komendę (i nacisnąć enter):

```
git clone SKOPIOWIANY_URL
```

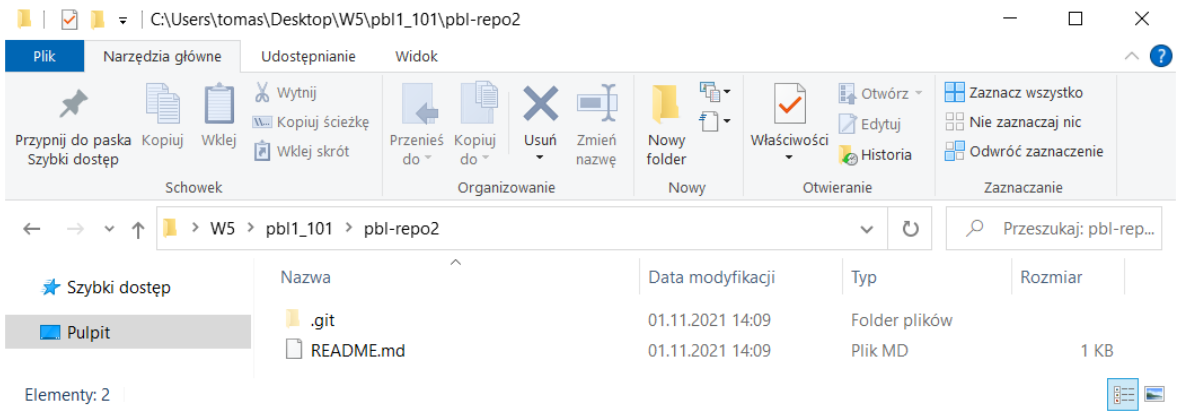
gdzie: SKOPIOWANY_URL np.

```
git clone https://github.com/TomaszMrozekPBL/pbl-repo2.git
```

Poniżej przedstawiono zrzut ekranu z informacją o poprawnym skopiowaniu repozytorium.



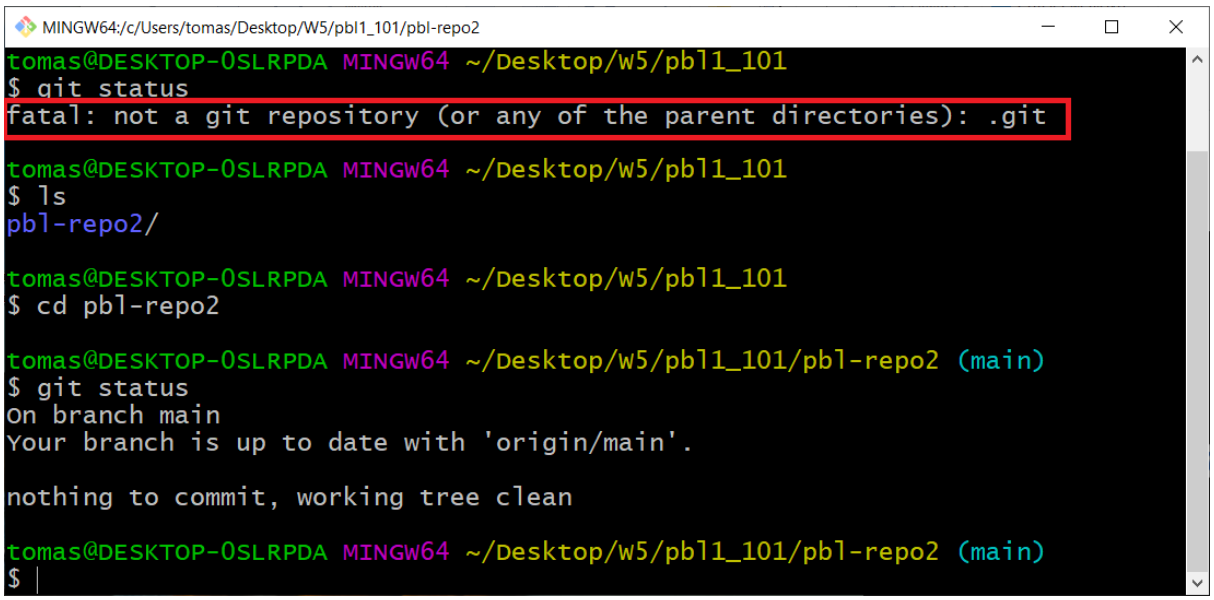
W folderze „pbl1_101" został utworzony folder „pbl-repo2" (nazwa repozytorium zdalnego), natomiast w folderze tym powinien znajdować się plik „README.md". Dodatkowo znajduje się tam również ukryty folder „**.git**", który zawiera wszystkie niezbędne informacje do poprawnego działania repozytorium.



Aby sprawdzić informacje o sklonowanym repozytorium należy użyć komendy:

git status

Aby użyć tej komendy, należy znajdować się w folderze z repozytorium. W przeciwnym otrzymamy błąd:



Aby wyświetlić foldery, jakie znajdują się w danym folderze należy użyć komendy „ls”. Wyświetlą się nazwy folderów, w tym przypadku „pbl-repo2”, który jest folderem repozytorium. Aby wejść do tego folderu należy wpisać komendę „cd pbl-repo2”. Następnie można sprawdzić status repozytorium wpisując „git status”.

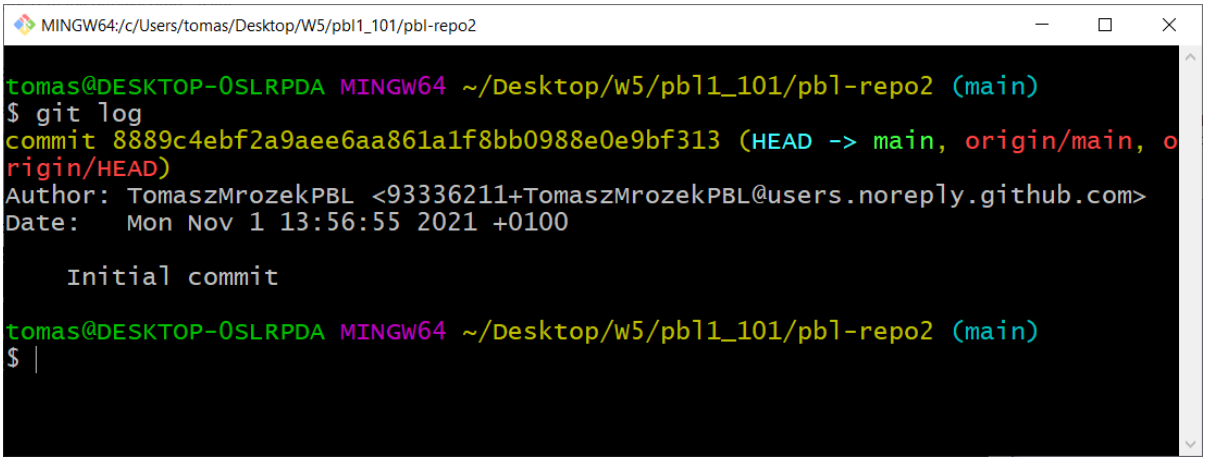
```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Z powyższych informacji można odczytać następujące informacje:
- jesteś w gałęzi głównej „main branch”,
- twoja gałąź jest aktualna względem oryginalnej,
- nie ma żadnych plików do „zakomitowania”, brak jest jakichkolwiek zmian.

Kolejną wartą uwagi komendą jest „git log”.

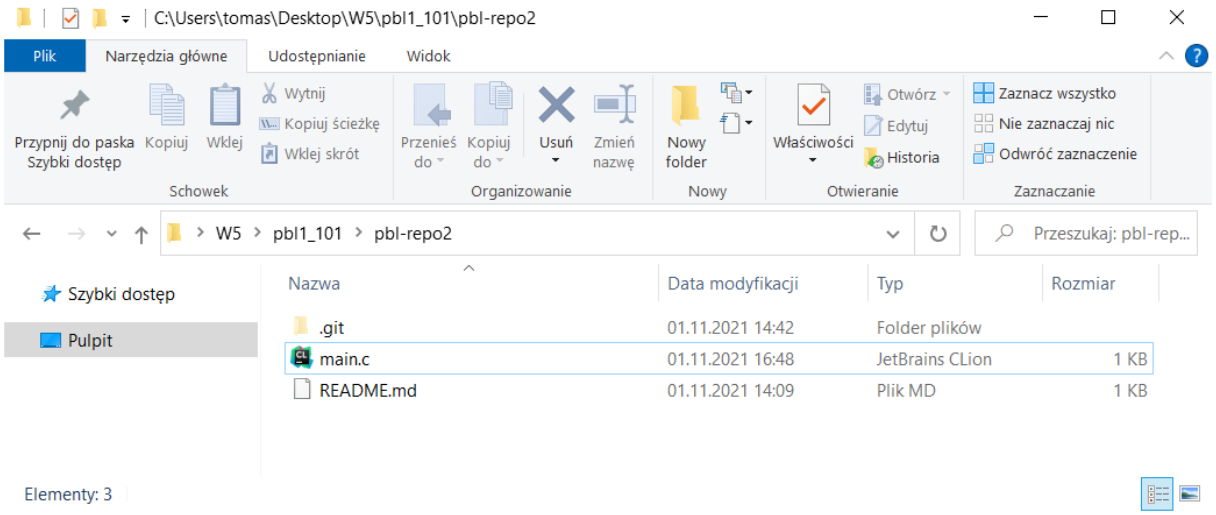
git log



Wyświetla ona listę commitów. W tym przypadku widać commit początkowy, moment utworzenia repozytorium. Dodatkowo widać autora, adres mailowy oraz czas. Należy jeszcze wspomnieć, czym jest **HEAD**. Jest to etykieta oznaczająca „aktualny" commit, czyli ten, który będzie rodzicem dla nowo utworzonego (po jego utworzeniu HEAD przesunie się automatycznie).

c) Commit

Do przeprowadzenia operacji „commit" zostanie wykorzystany następujący przykład. W sklonowanym folderze ze zdalnego repozytorium (tutaj folder: pbl-repo2) zostanie stworzony plik **main.c**

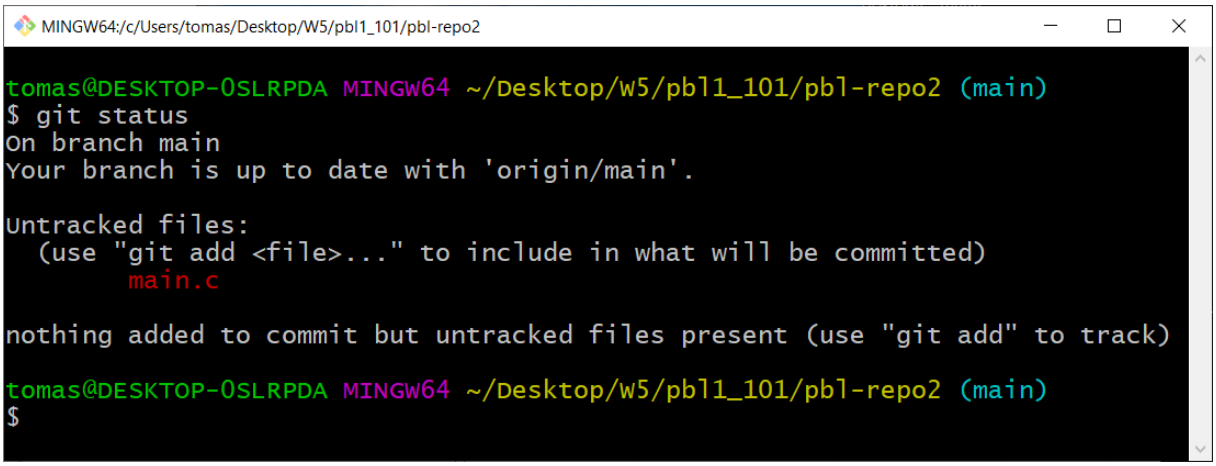


o następującej zawartości:

```
#include<stdio.h>

int main() {
        puts("Hello world");
        return 0;
}
```

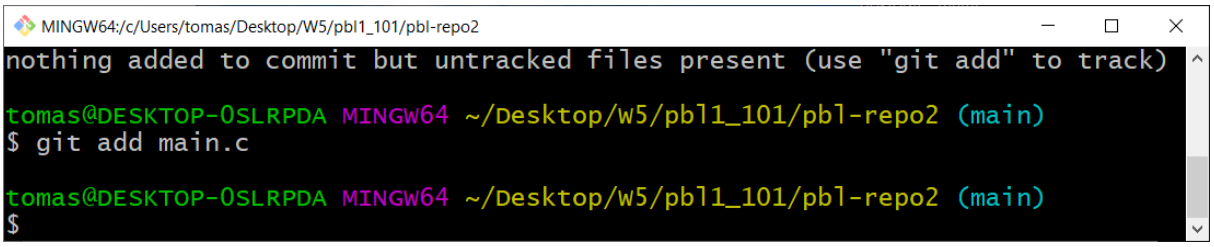
Po stworzeniu pliku i dodaniu zawartości należy sprawdzić w celach treningowych status wykorzystując znaną już komendę „git status”.



Z otrzymanego komunikatu można zobaczyć zaznaczony na czerwono plik main.c oraz informacje o tym, że jeden z plików nie jest śledzony przez GIT’a. Oznacza to, że GIT nie będzie się interesował tym plikiem. Aby wskazać, że nowy plik jest częścią projektu, należy wpisać następującą komendę:

```
git add main.c
```

Po wykonaniu komendy nie zostanie wyświetlona żadna dodatkowa informacja.



Warto jednak sprawdzić status zmian, wpisując w konsoli znaną już komendę „git status”.

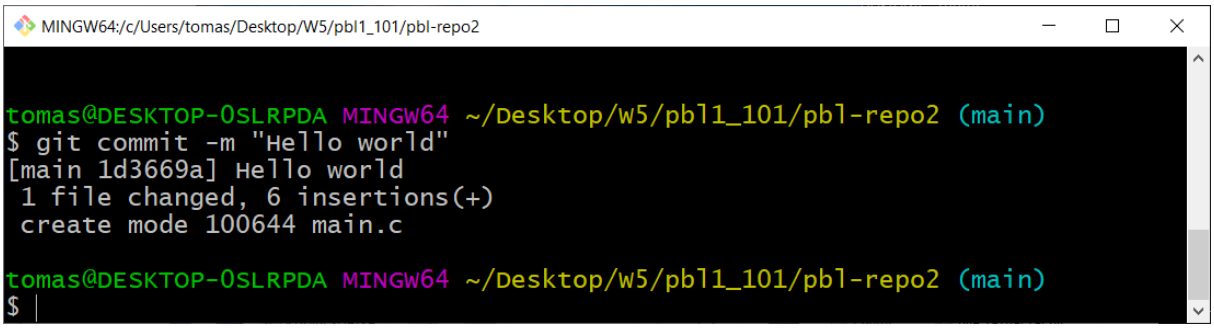


Widać informację kolorem zielonym, o nowym pliku „new file: main.c”. Plik „main.c” został dodany do indeksu – jest to miejsce, w którym przygotowywany jest commit. Jest to pierwszy

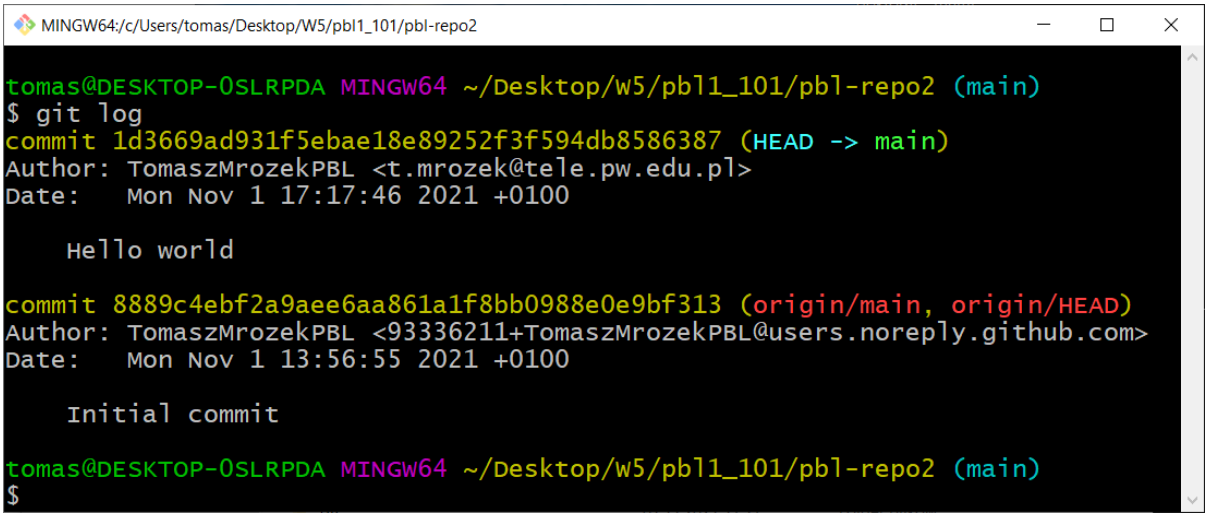
etap procesu. Kolejnym etapem jest wykonanie komendy „commit”. Flaga -m oznacza ustawienie komentarza opisującego zmiany w projekcie w tym przykładzie „Hello world”.

```
git commit -m "Hello world"
```

Po poprawnym „zacommitowaniu” w konsoli wyświetlona zostanie poniższa informacja:



Wykorzystując komendę „git log” można sprawdzić historię „commit’ów” w tym informacje o ostatnim.



d) Następny commit

Jak wygląda sytuacja w przypadku kolejnego commit’a? Commit’y można dodawać w kilku plikach jednocześnie.

W pliku main.c dodano jeden wiersz kodu:

```
#include<stdio.h>

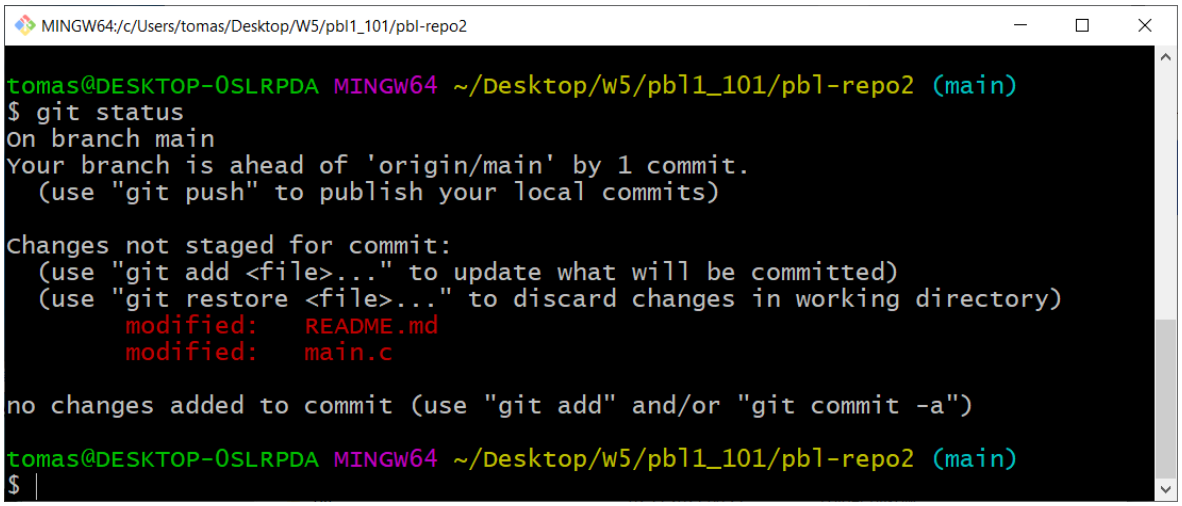
int main() {
        puts("Hello world");
        puts("Second commit, example");
        return 0;
}
```

Natomiast w pliku README.md komentarz.

```
# pbl-repo2

Sample file for commit testing.
```

W konsoli wpisujemy komendę „git status”.



Widać dwie informację. W pierwszej części komentarza jest informacja o tym, że został wykonany jeden commit, jednak nie został on wysłany do repozytorium zdalnego. Można to zrobić wykonując komendę:

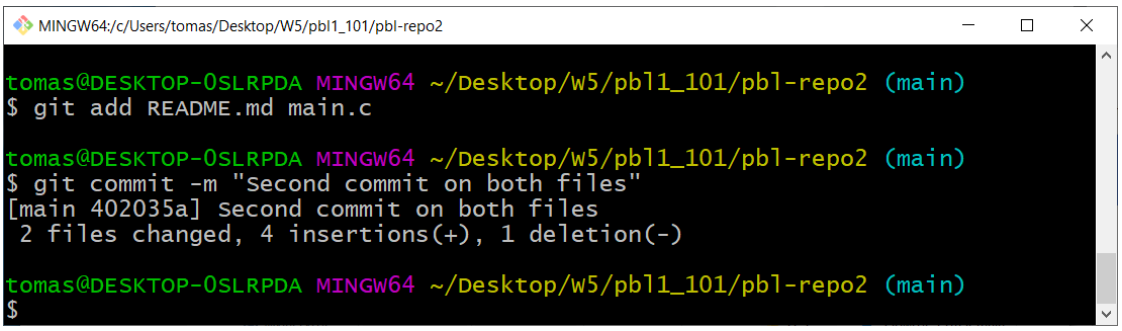
```
git push
```

W drugiej części widać informację o modyfikacji dwóch plików README.md oraz main.c, zgodnie z wykonanymi operacjami na plikach. Widać, że GIT rozpoznał zmiany w śledzonych plikach. Aby wykonać drugi commit należy powtórzyć procedurę taką, jak przy pierwszym commit. Pliki należy dodać do indeksu znaną komendą:

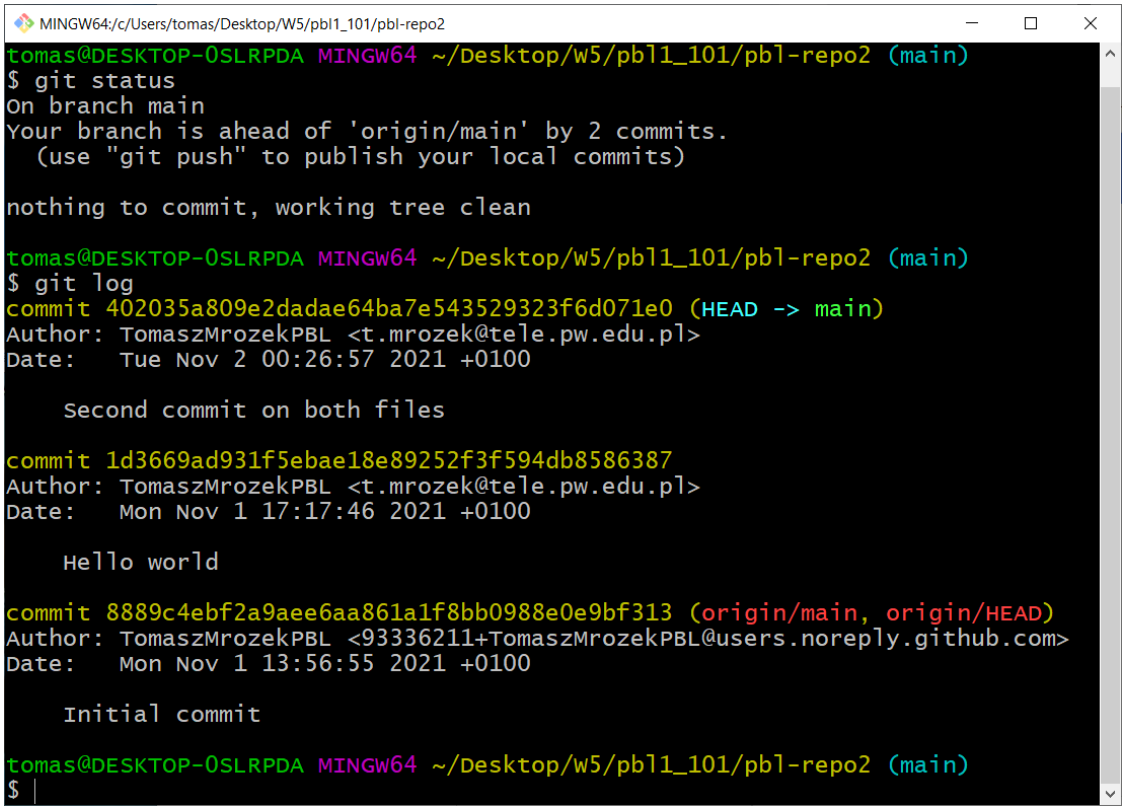
```
git add README.md main.c
```

Warto zwrócić uwagę, że po spacji można dodawać kilka plików na raz. W kolejnym etapie należy wykonać polecenie commit:

```
git commit -m "Second commit on both files"
```



Dla sprawdzenia „git status” i „git log”



PODSUMOWANIE !!

Aby wykonać commit'a należy wykonać po sobie dwie komendy.

```
git add file_name1 file_name2 … file_nameN
git commit -m "Commit information"
```

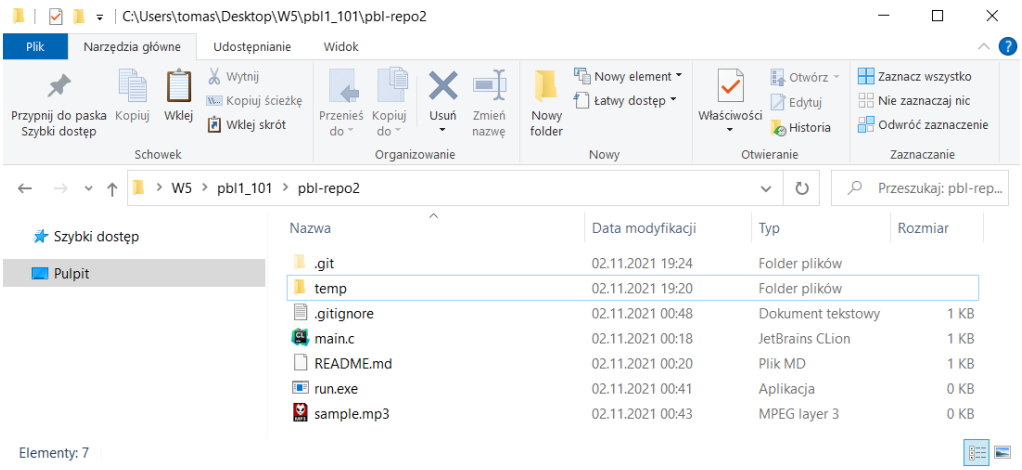
Jeśli chcemy przesłać zmiany na serwer zdalny należy wykonać polecenie

```
git push
```

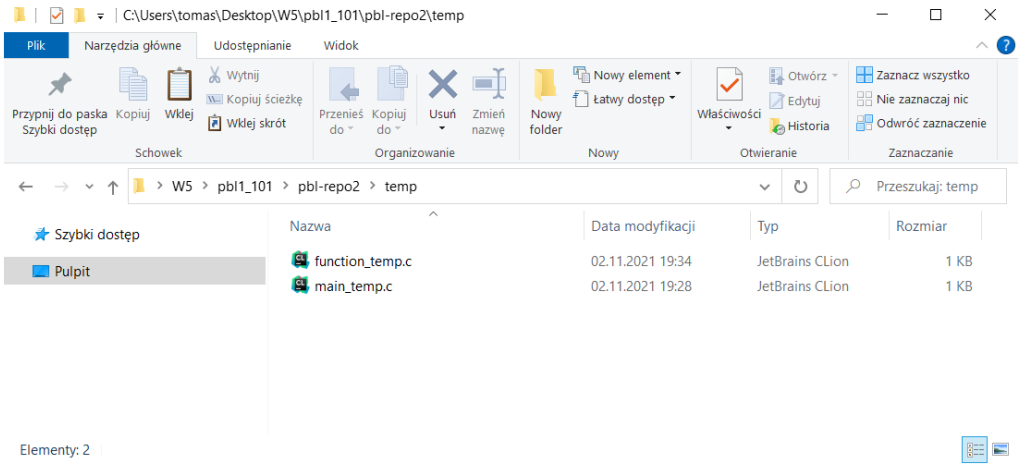
Polecenie to zostanie przedstawione nieco później.

e) Dodawanie całych folderów oraz wycofywanie zmian

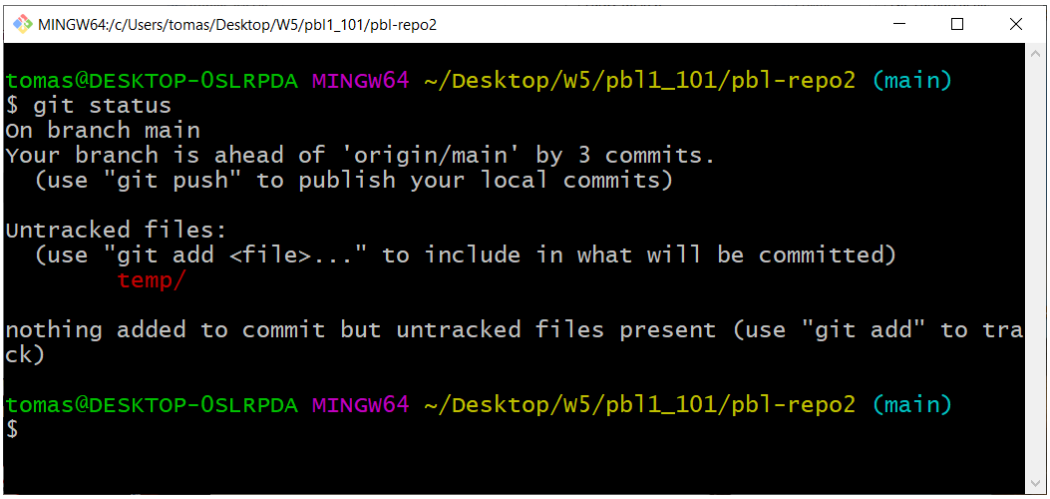
Możliwe jest dodawanie cały folderów z zawartością (plikami w środku). Procedura nie różni się niczym od dodawania poszczególnych plików. Dla przykładu stworzony został folder „temp",



a w nim dwa pliki „function_temp.c” oraz „main_temp.c”.



Korzystając z komendy „git status” widać, że GIT wykrył nieśledzony folder „temp/”.



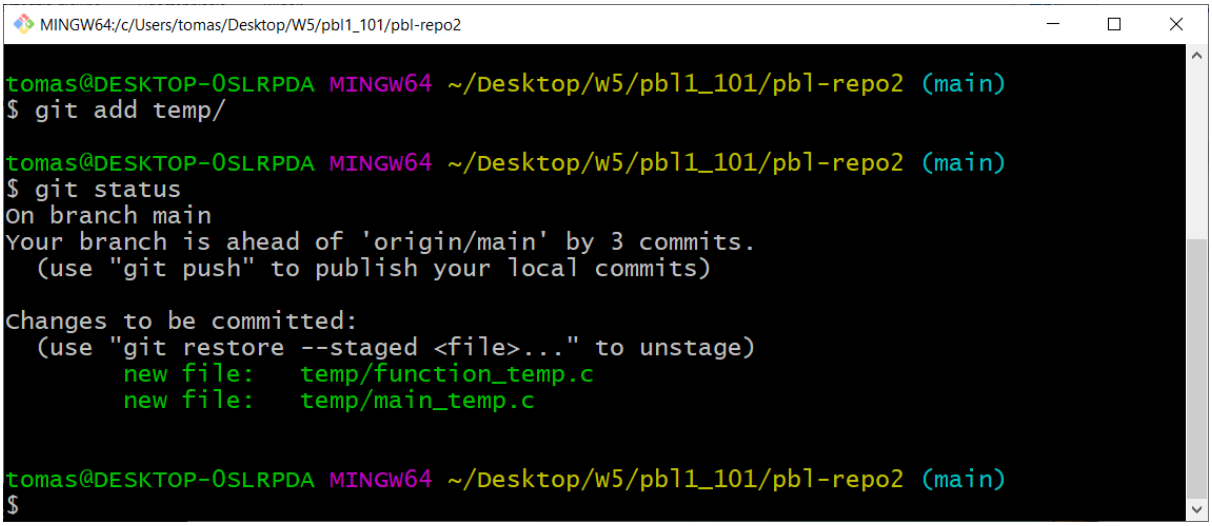
W celu dodania folderu „temp” wraz z zawartością należy w konsoli wpisać komendę:

```
git add temp
```

lub równoważnie

```
git add temp/
```

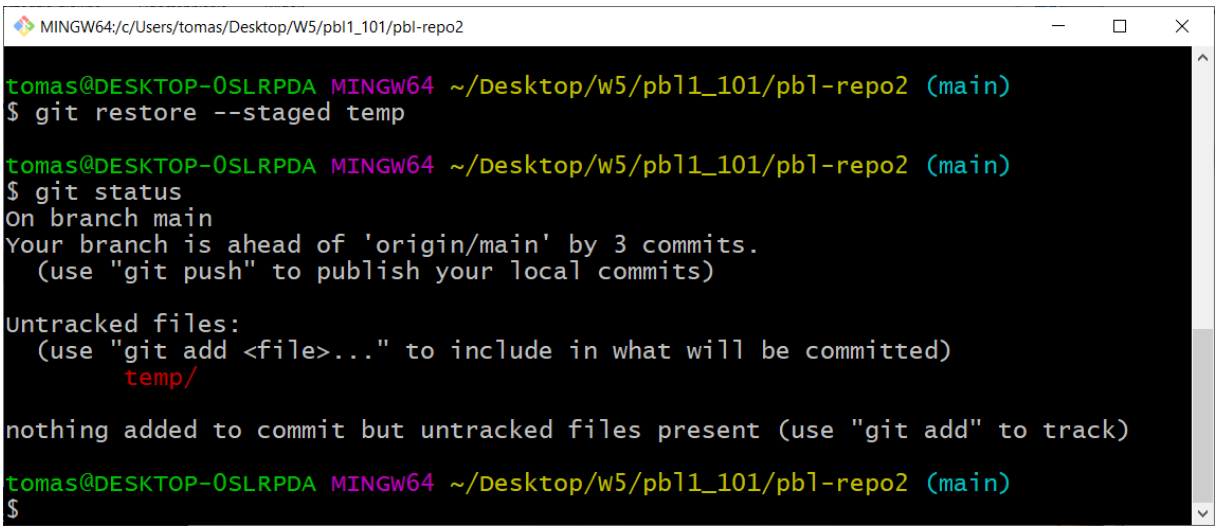
Wpisując komendę „git status” widać, że GIT dodał do śledzenia dwa nowe pliki znajdujące się w folderze „temp”.



Zmiany można cofnąć. Aby usunąć folder „temp” ze śledzonej listy należy wpisać komendę:

```
git restore --staged temp
```

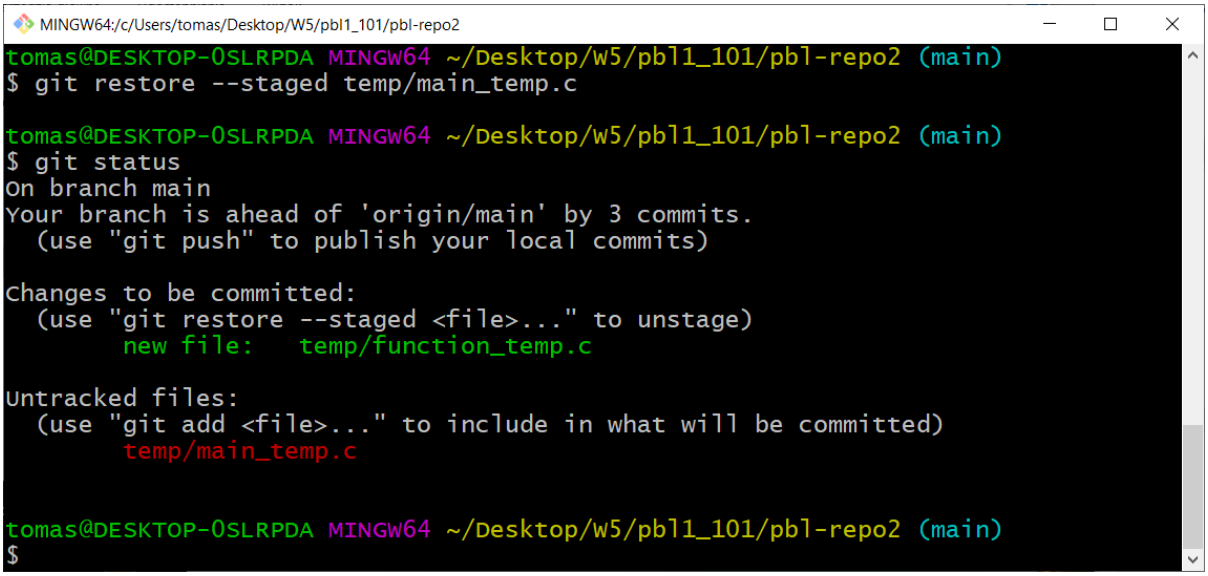
Wpisując komendę „git status” widać, że GIT ponownie wykryje folder „temp” jako nieśledzony.



Zamiast całego folderu można usunąć śledzenie wybranego pliku np. main_temp.c, komenda poniżej:

```
git restore --staged temp/main_temp.c
```

W celu sprawdzenia zmian wpisujemy komendę „git status”:

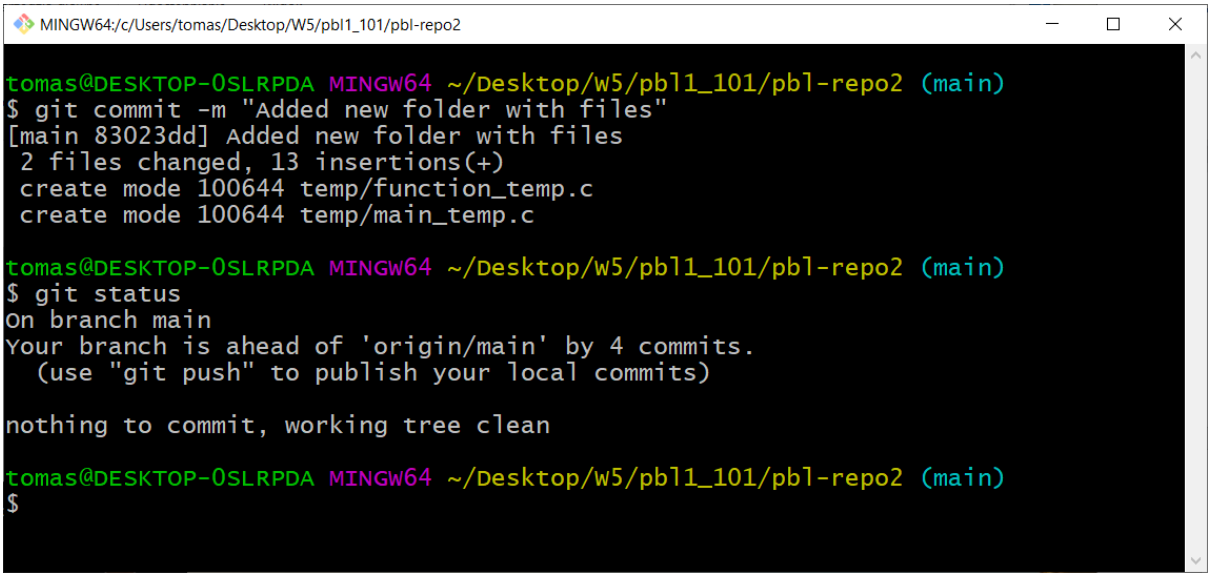


W tym przypadku widać, że GIT wykrył plik „main_temp.c” jako nieśledzony, natomiast drugi plik pozostał wykryty jako nowy, ale nie „zacommitowany”.

Wracając do głównego wątku dodawania całego folderu, należy wykonać drugą komendę tj. commit.

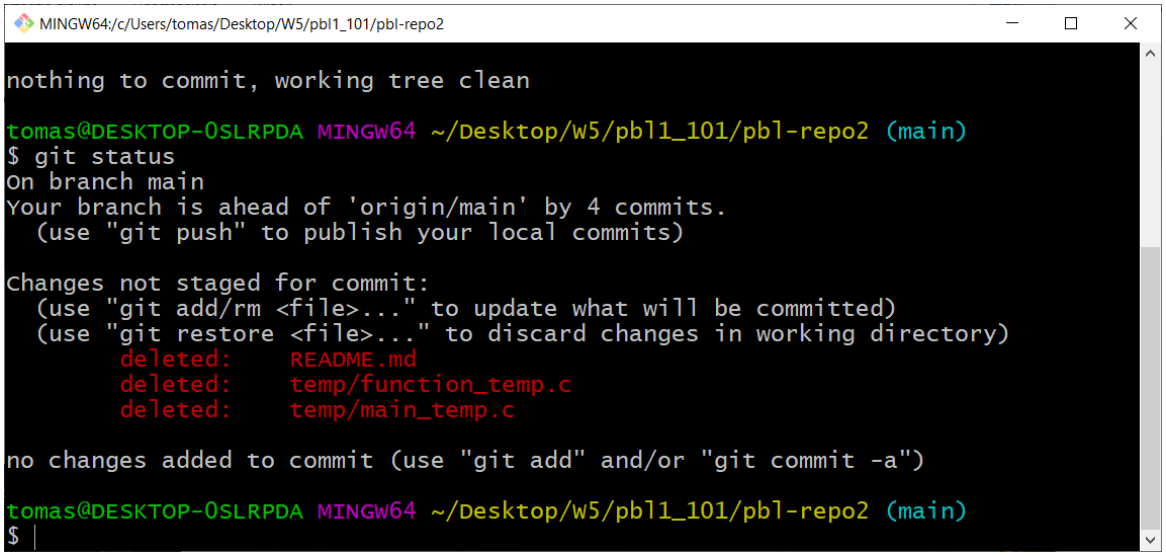
```
git commit -m "Added new folder with files"
```

Dla sprawdzenia poprawności dodania folderu wykonujemy komendę „git status”:



Commit został wykonany poprawnie. Dodano 2 nowe pliki znajdujące się w folderze. Należy pamiętać, że pliki wciąż znajdują się tylko w repozytorium lokalnym. Aby je przesłać na repozytorium zdalne należy użyć komendy „git push”.

Nie oznacza to jednak, że repozytorium lokalne jest bezużyteczne. Aby to wykazać usuń z folderu projektowego folder „temp” oraz np. plik „README.md”. Następnie sprawdź status repozytorium (git status):



Widać, że z repozytorium zostały usunięte pliki z folderu temp oraz plik README.md (czerwona czcionka w konsoli). Dzięki repozytorium możesz jednak przywrócić te pliki. Wystarczy, że w konsoli wpiszesz komendę:

```
git restore temp
```

jeśli chcesz przywrócić folder „temp” wraz ze wszystkimi plikami.

```
git restore temp/main_temp.c
```

jeśli chcesz przywrócić wybrany plik z folderu (tutaj plik „main_temp.c” z folderu temp).

```
git restore README.md
```

jeśli chcesz przywrócić plik README.md z głównego folderu.

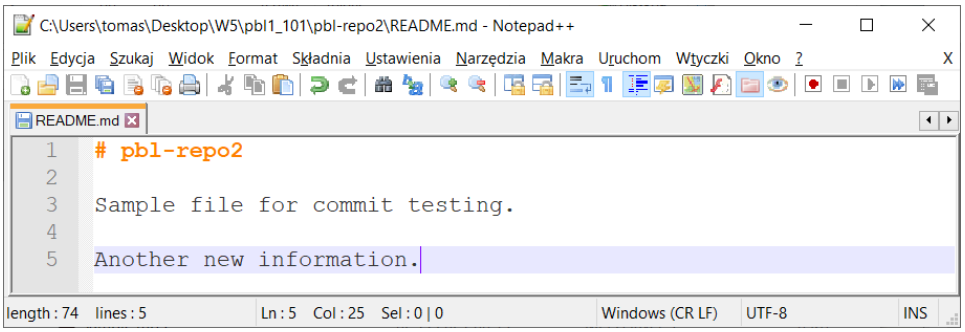
Aby przywrócić plik i folder wraz z plikami za pomocą jednej komendy:

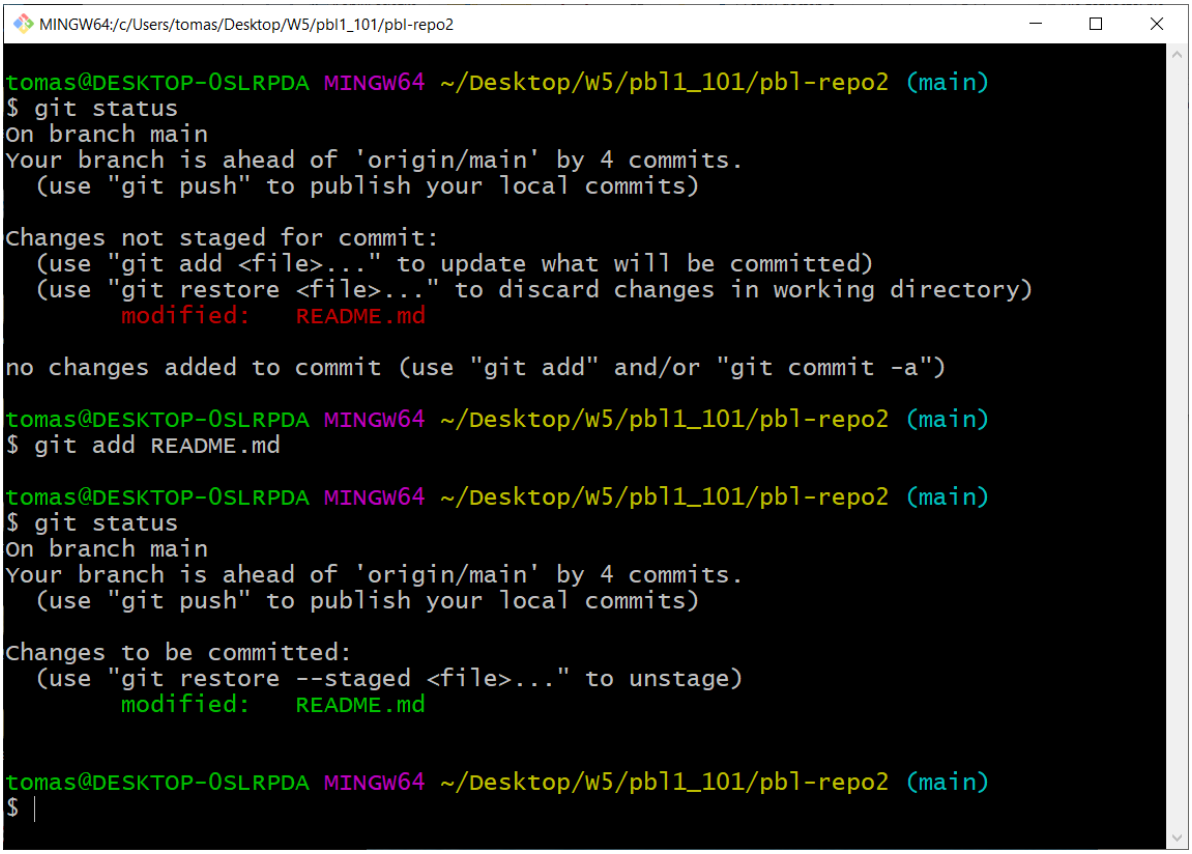
```
git restore README.md temp
```

Przywracać można również z wykorzystaniem innej komendy:

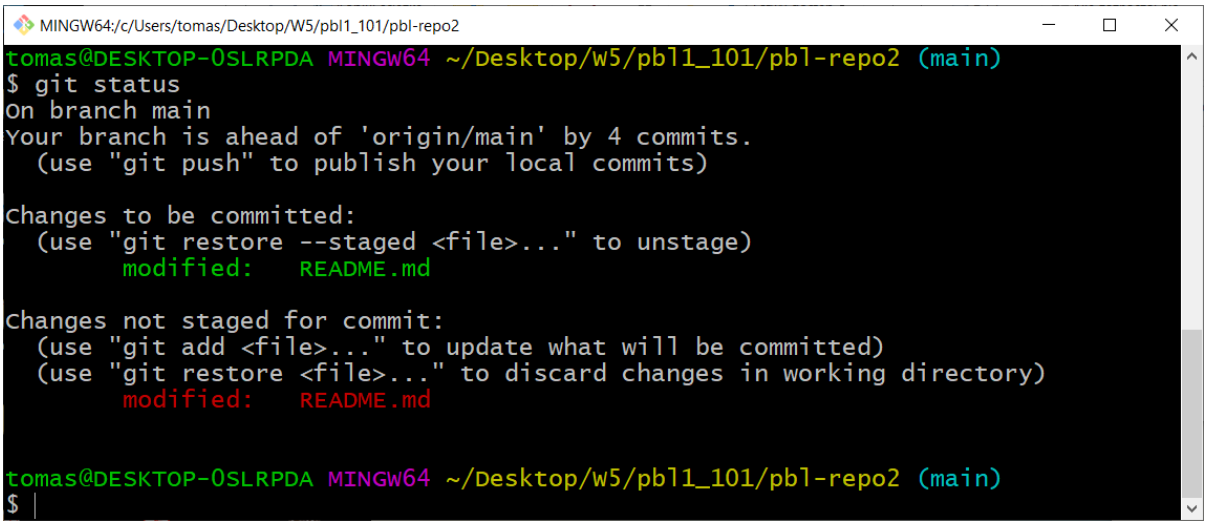
```
git checkout -- README.md
```

Co, w przypadku, jeśli wprowadzimy zmiany w pliku i dodamy go do indeksu (komenda: git add file_name), ale nie chcemy go ostatecznie commitować?

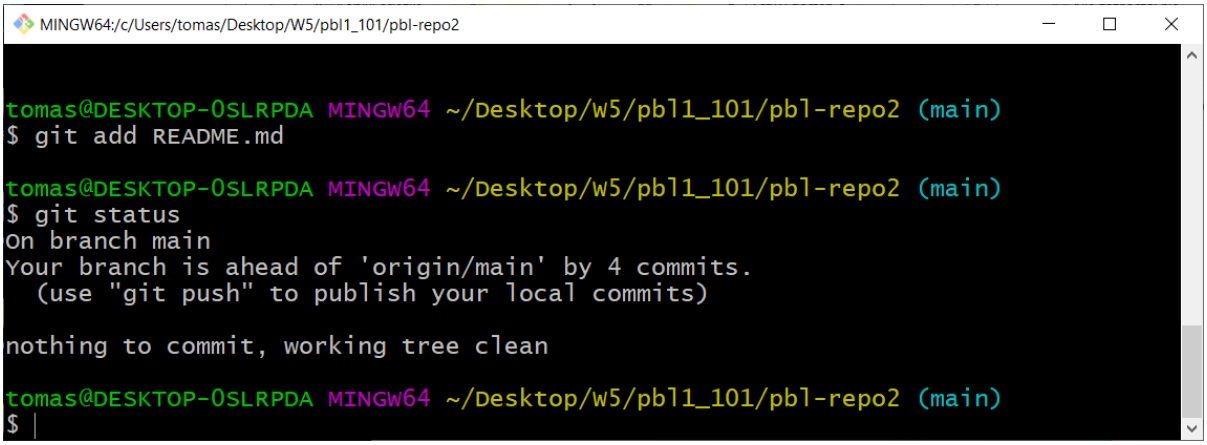




Wracamy do pliku README.md i usuwamy dodany komentarz „Another new information.” Zapisujemy plik i wyświetlamy status repozytorium.



Następnie dodajemy nowe zmiany do indeksu: „git add README.md” i sprawdzamy status:

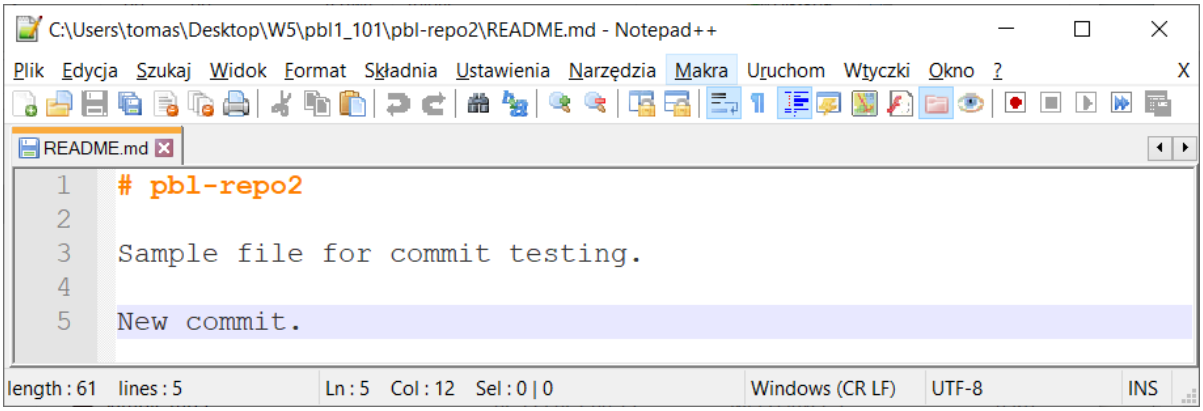


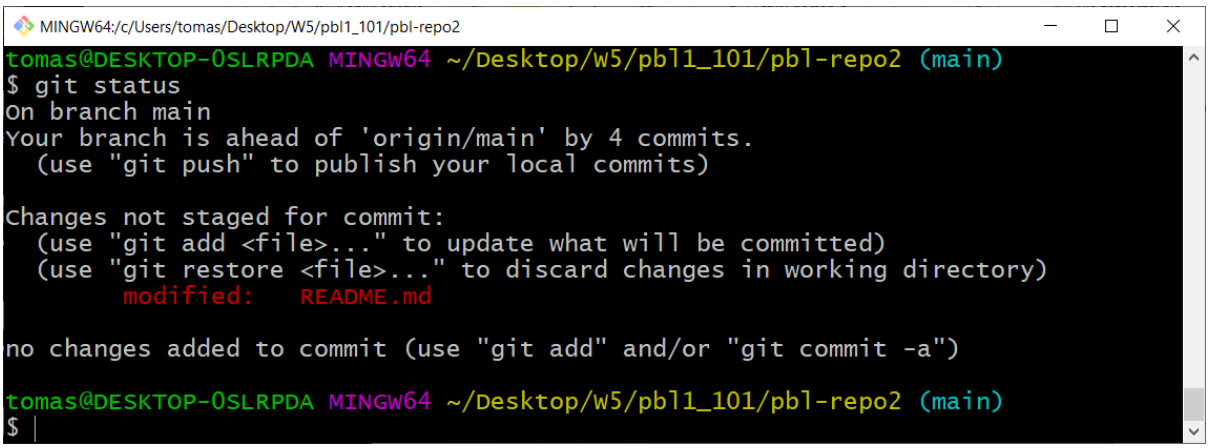
Widać, że po powrocie do poprzedniego stanu (usunięciu informacji z pliku README.md) GIT nie widzi nowych zmian w pliku i commit nie jest potrzebny. Podsumowując, wystarczy ręcznie usunąć ostatnio dodane informacje i dodać dany plik do indexu (git add nazwa_pliku), a GIT rozpozna status pliku jako aktualny.

Opisane powyżej modyfikacje miały status „dokonywania zmian" innymi słowy „nadpisywania" nowych stanów pliku. Jeśli nie chcemy nadpisywać tylko cofnąć zawartość pliku do stanu poprzedniego należy wpisać komendę:

```
git restore README.md
```

Dla przetestowania dodaj zmianę w pliku README.md:





następnie:

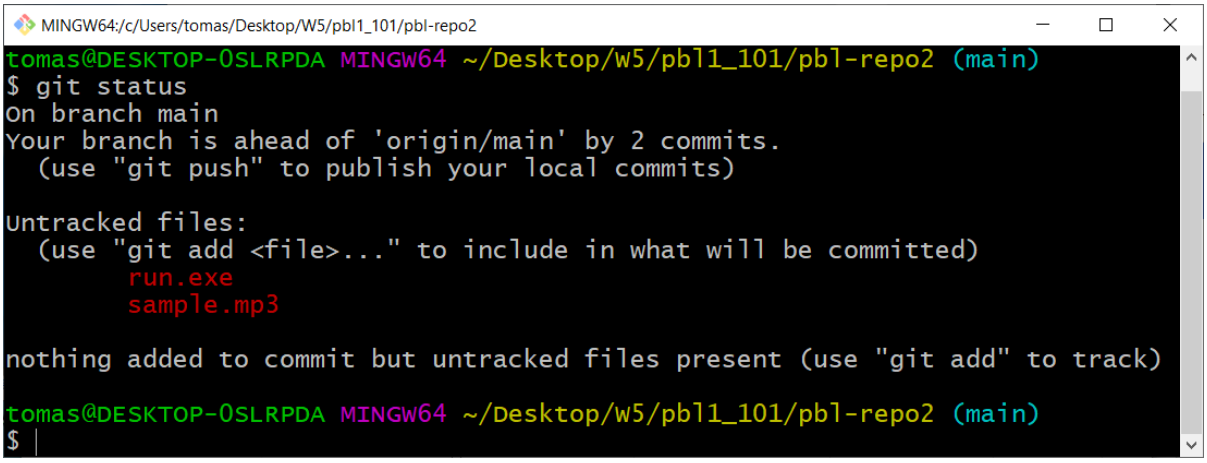
```
git restore README.md
```

Zawartość pliku wróci do stanu poprzedniego. Z jego zawartości zniknie komentarz „New commit.”

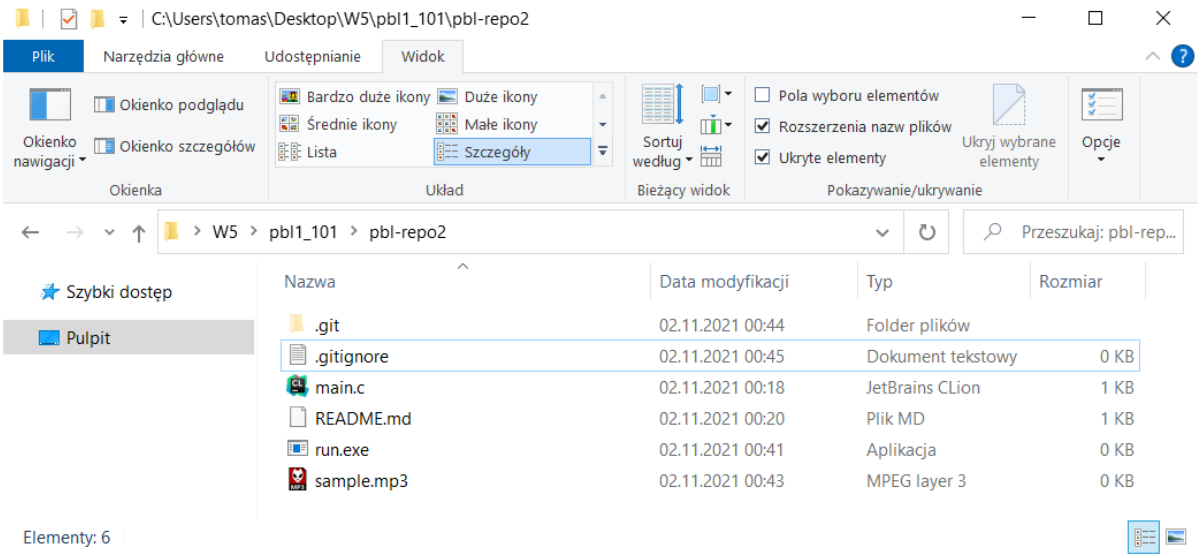
f) Ignorowanie wybranych plików przez GIT

W trakcie pracy zwłaszcza programistycznej często dzieje się tak, że w trakcie kompilowania kodu tworzone są dodatkowe pliki pomocnicze/wynikowe. Repozytorium jest wykorzystywane przede wszystkim do plików źródłowych. GIT jednak będzie rozpoznawał wszystkie pliki i o nich przypominał. Aby jednak ignorować wybrane rozszerzenia plików można stworzyć specjalny plik .gitignore, w którym umieszcza się „niechciane” rozszerzenia, które mają być niewidoczne dla GIT’a.

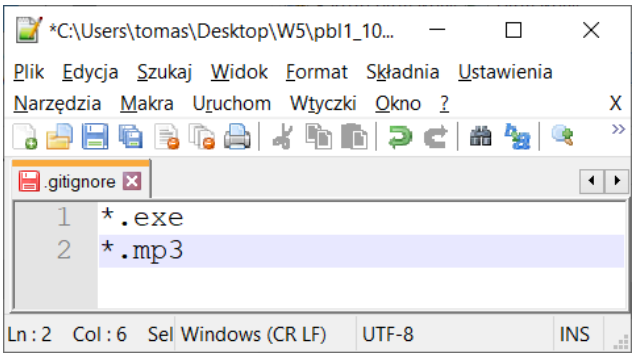
Dla przykładu w folderze repozytorium „pbl-repo2” utworzono pliki z rozszerzeniem run.exe oraz sample.mp3. Po wpisaniu w konsoli komendy „git status” widać, że pliki zostały wykryte jako nieśledzone.



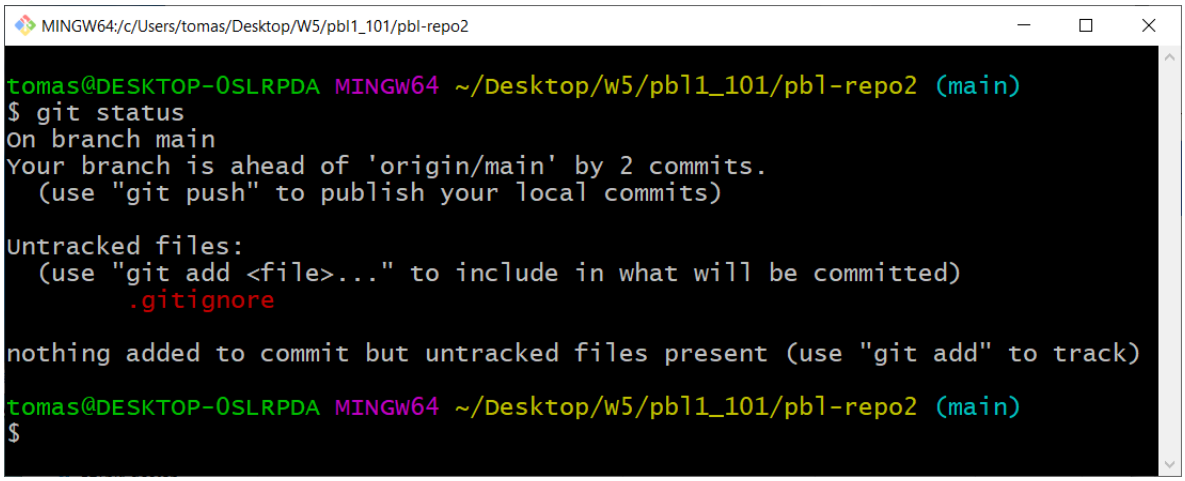
Aby GIT ignorował te pliki należy utworzyć plik bez rozszerzenia o nazwie .gitignore.



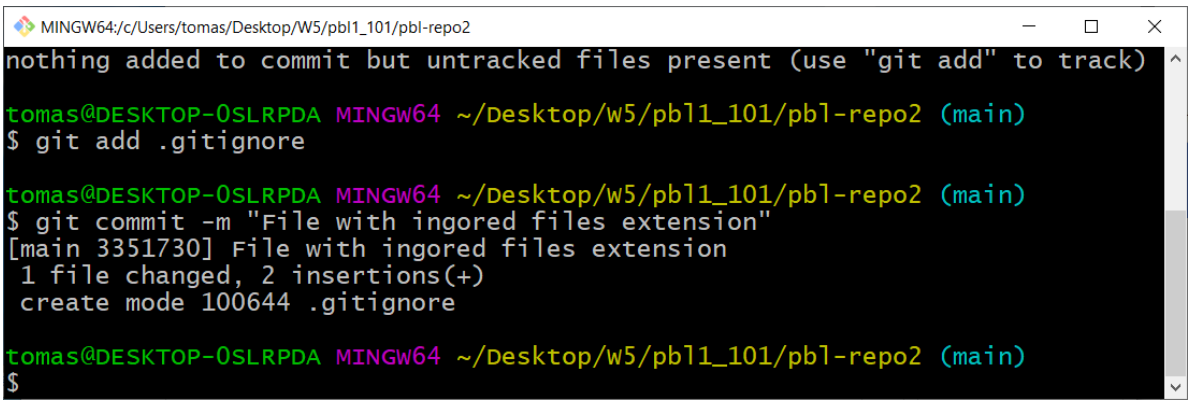
W pliku .gitignore umieszczono rozszerzenia, które GIT powinien ignorować:



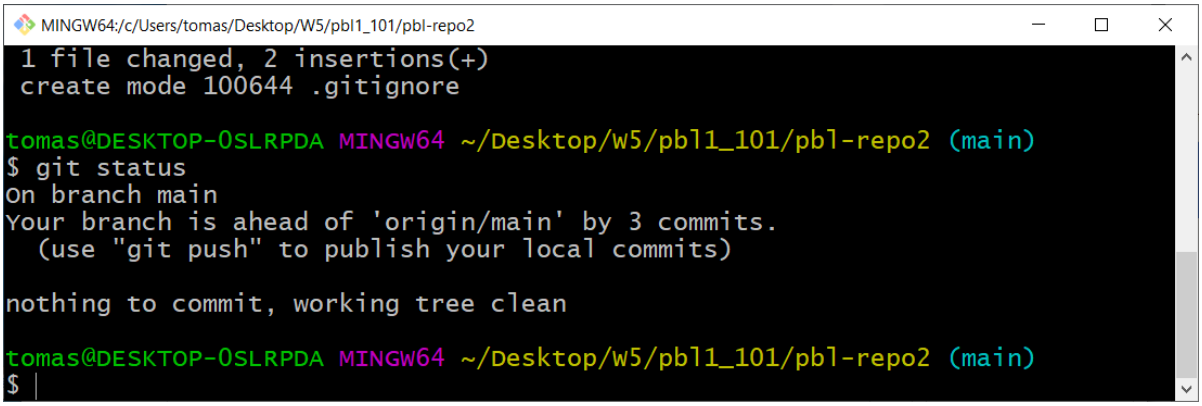
Aby sprawdzić poprawność ignorowania wybranych plików przez GIT, należy wybrać komendę „git status”:



Pliki ze wskazanym rozszerzeniem są ignorowane. Widać natomiast informacje o pliku .gitignore i ten plik warto „zacomittować”.

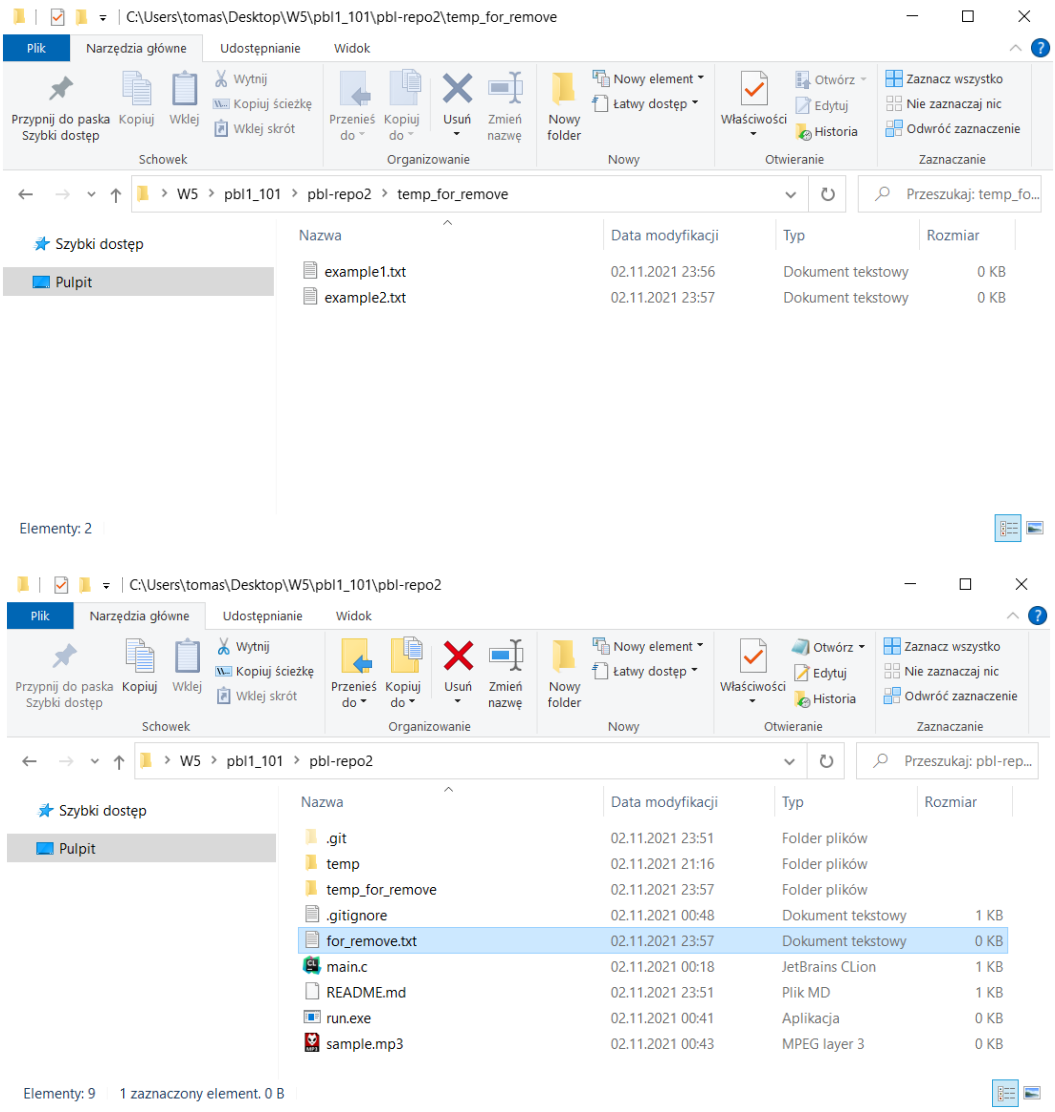


Dla sprawdzenia „git status”

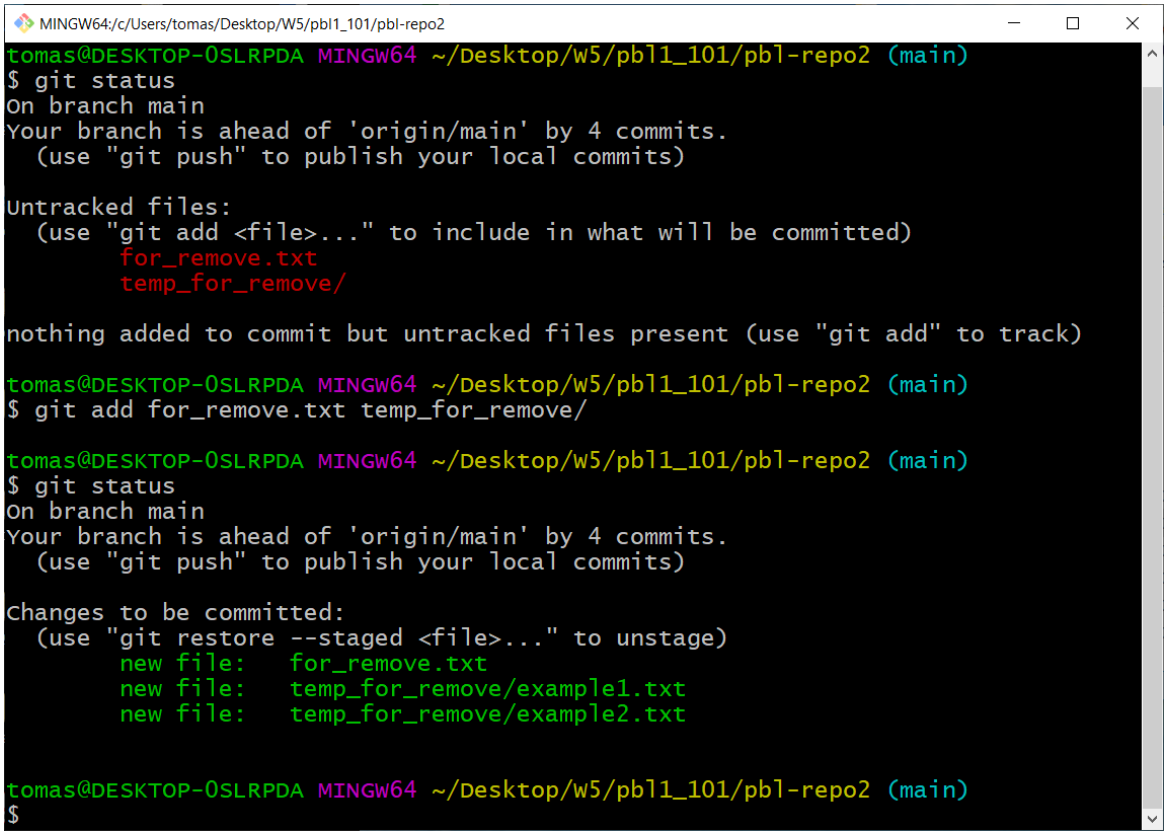


g) Usuwanie plików z repozytorium

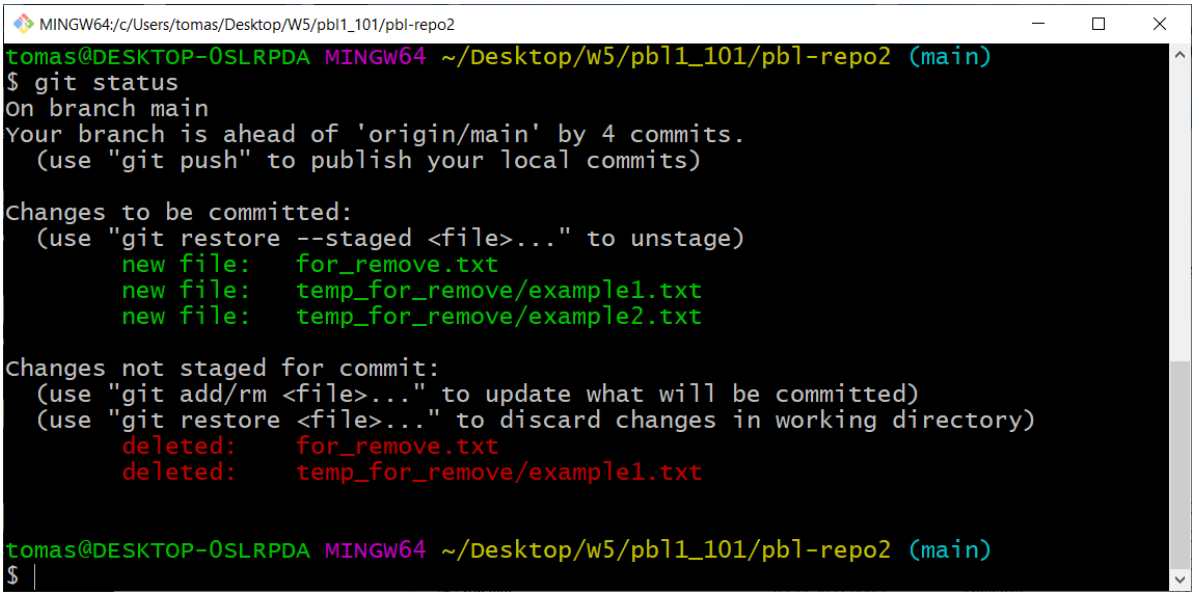
W jaki sposób usuwać wybrane pliki z repozytorium? Dla przykładu utworzono nowy folder z dwoma plikami w środku (folder o nazwie „temp_for_remove” oraz dwa puste pliki „example1.txt” i example2.txt” oraz dodatkowy plik w katalogu głównym („for_remove.txt”).



Nowe dane dodano do indeksu i za commitowano.

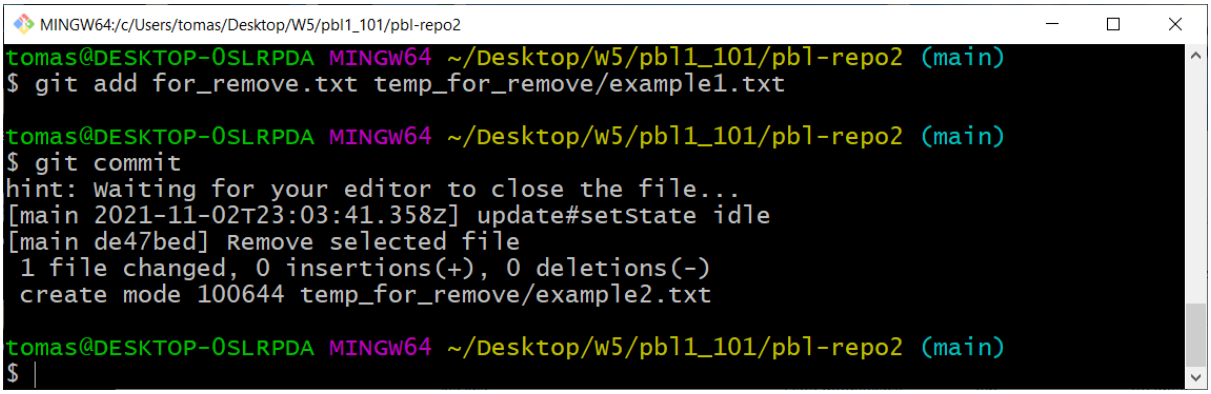


Chcemy usunąć plik „for_remove.txt” oraz „example1.txt” z folderu „temp_for_remove”. Usuwamy pliki ręcznie i sprawdzamy status repozytorium:



Aby usunąć pliki wykonujemy dwie znane instrukcje z commit tzn:

```
git add for_remove.txt temp_for_remove/example1.txt
git commit
```



Warto zauważyć, że przy komendzie „git commit” nie podano komentarza (informacji o usunieciu). W takim przypadku zostanie uruchomiony preferowany edytor tekstu (tutaj MS Visual Studio), w którym zostaniemy poproszeni o wpisanie komentarza. Po jego wpisaniu commit zostanie wykonany zgodnie z powyższym zrzutem ekranu.

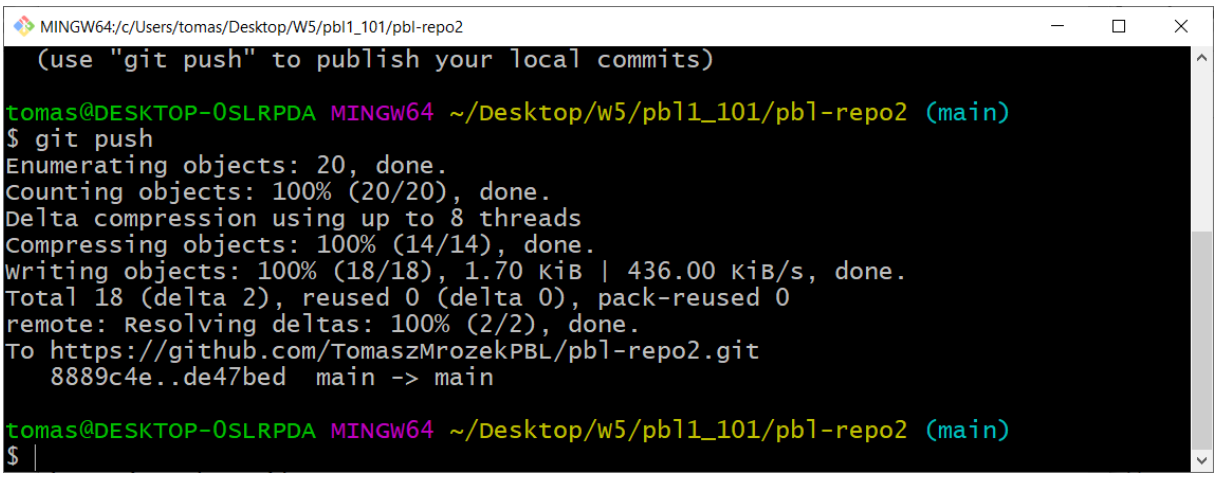
h) Synchronizacja ze zdalnym repozytorium

Wszystkie powyższe operacje były robione w repozytorium lokalnym. Aby przekazać zmiany z repozytorium zdalnego na lokalny należy wykonać polecenie:

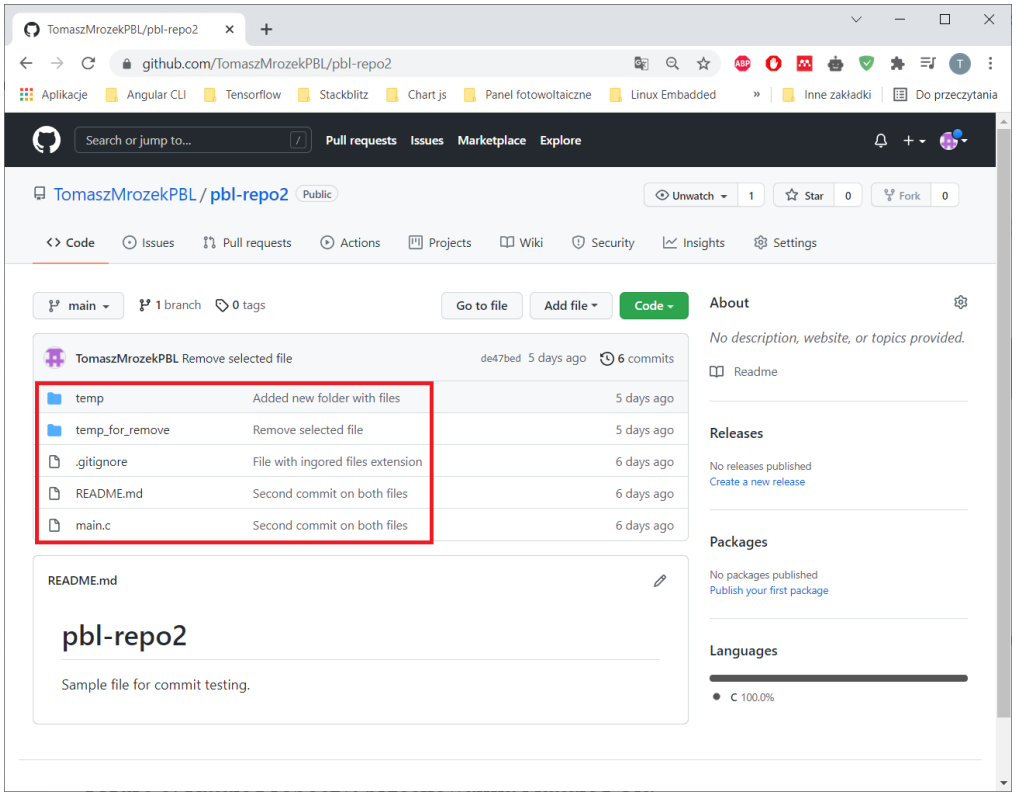
```
git push
```

Po wykonaniu tej komendy, Git może zażądać logowania do Gita zdalnego. Należy wtedy się zalogować na dane do utworzonego wcześniej konta w Git.

Po poprawnym wykonaniu komendy „git push” w konsoli wyświetlą się następujące informacje:

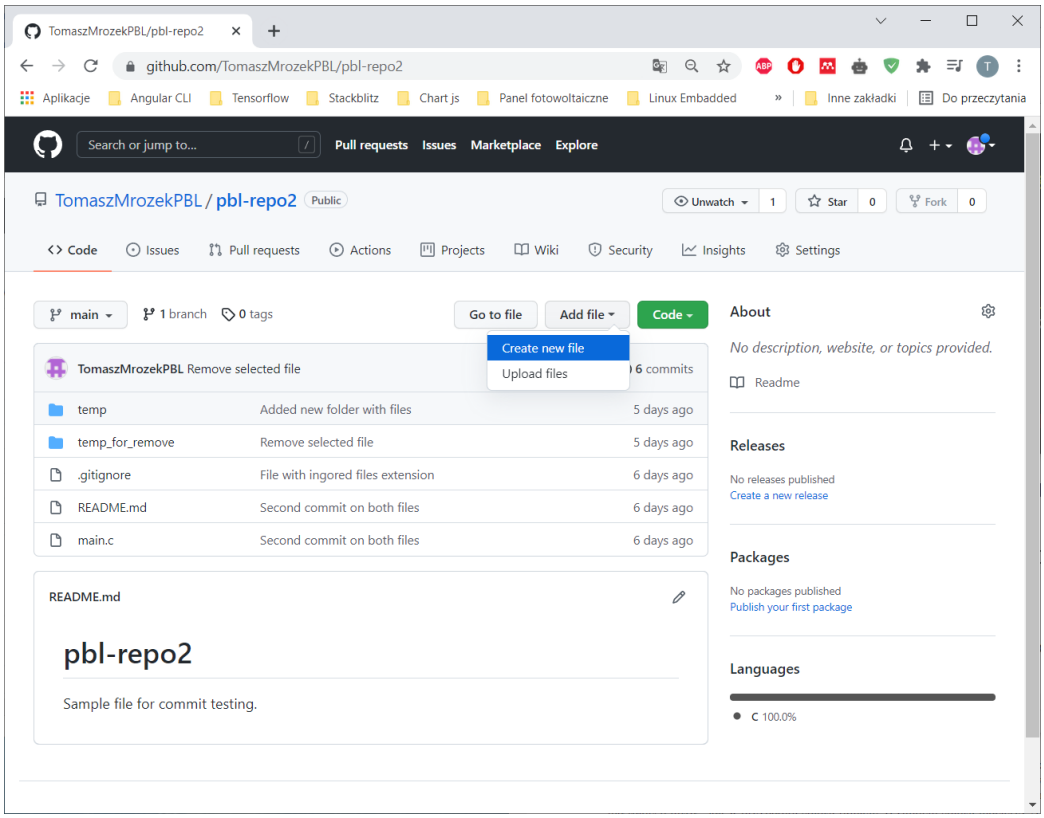


Aby obejrzeć zmiany należy zalogować się w przeglądarce do Git’a zdalnego lub odświeżyć przeglądarkę (np. klawisz F5, jeśli jesteśmy już zalogowanie na Git’a), aby zobaczyć zmiany:

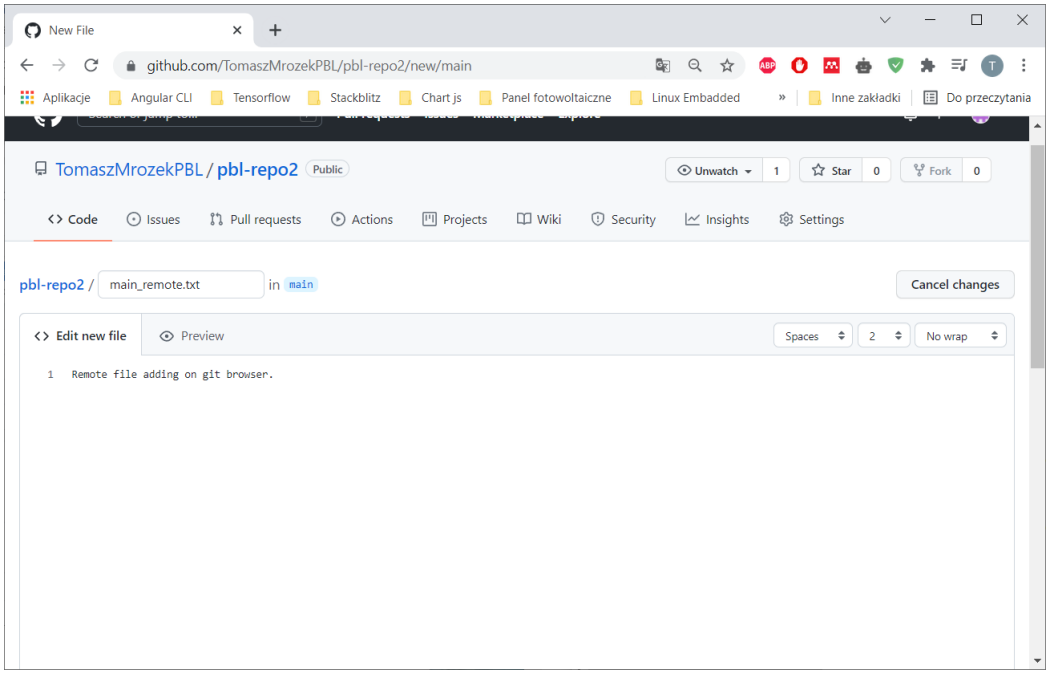


i) Tworzenie plików/folderów w repozytorium zdalnym

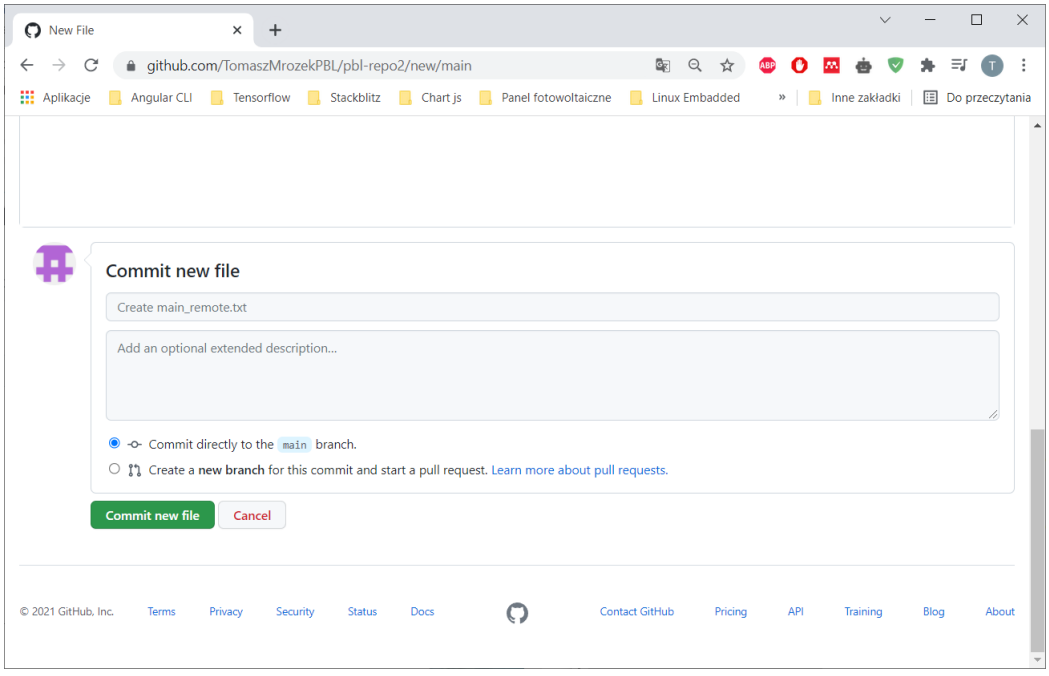
Pracując w repozytorium zdalnym można tworzyć pliki wykorzystując opcję „Add file” -> „Create new file” jak pokazano na zdjęciu poniżej.



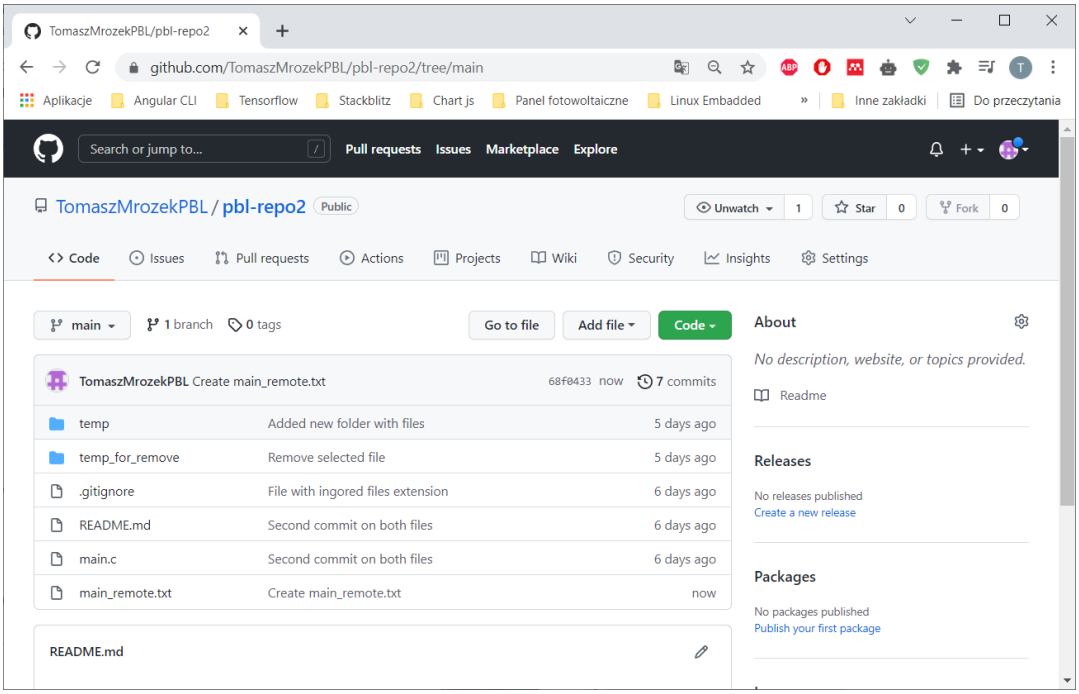
Na początku nadajemy nazwę pliku w tzw. textBox'ie obok „pbl-repo2/", a następnie w polu „< > Edit new file" należy wpisać zawartość pliku:



Następnie należy przewinąć stronę do dołu i kliknąć opcję „Commit new file" (wcześniej można dodać informacje do commita np. „Create main_remote.txt" i/lub dodatkowe informacje „Add an optional extended description..".



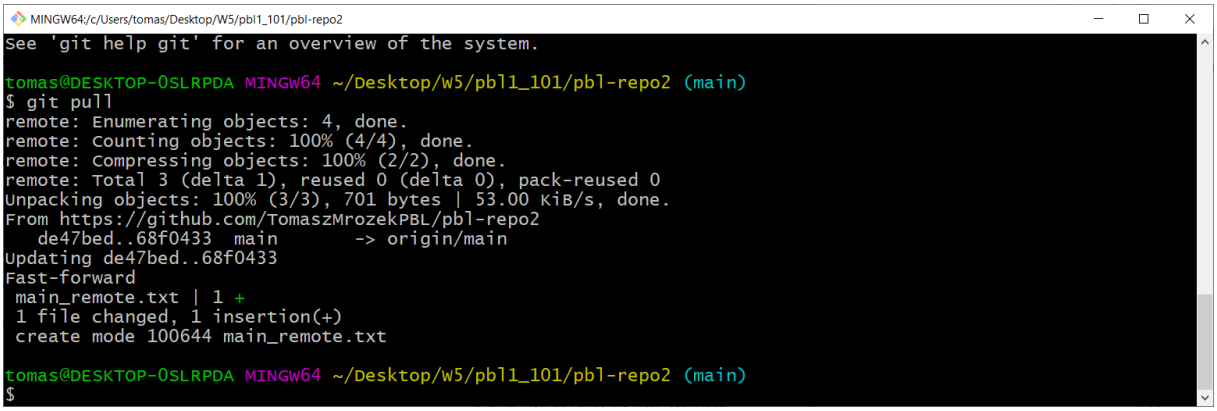
Po kliknięciu opcji „Commit new file" w widocznych plikach w repozytorium zdalnym zostanie utworzony i dodany nowy plik.



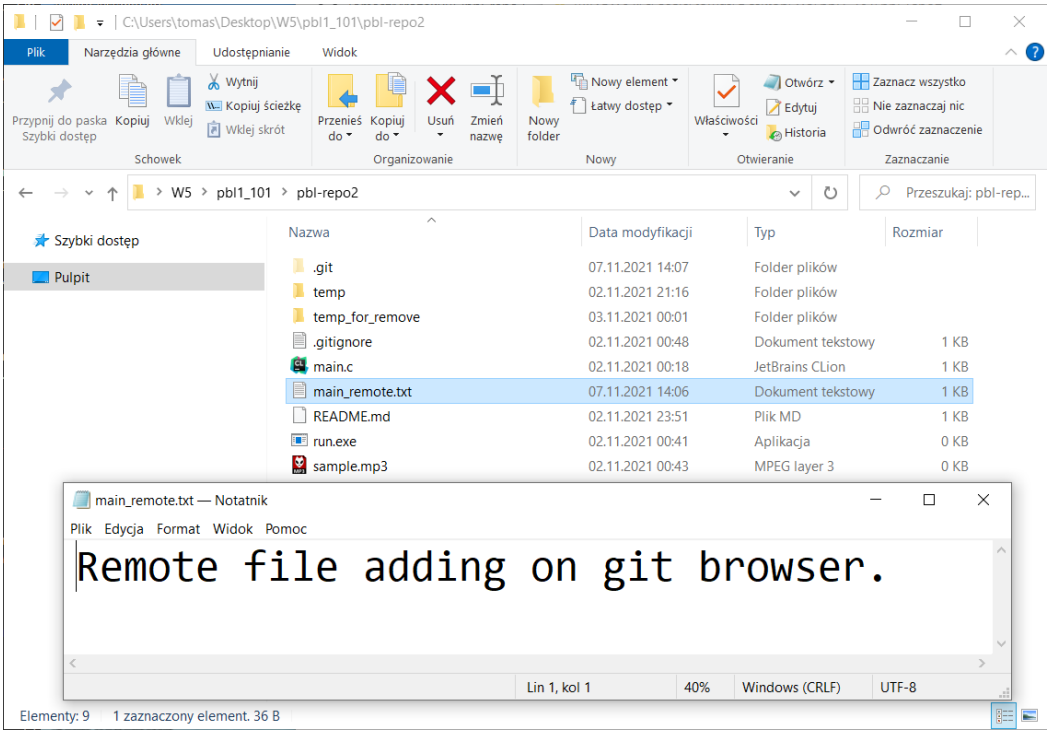
Plik został utworzony w repozytorium zdalnym. Aby uaktualnić repozytorium lokalne o nowo utworzony plik, należy przejść do konsoli i wpisać komendę:

```
git pull
```

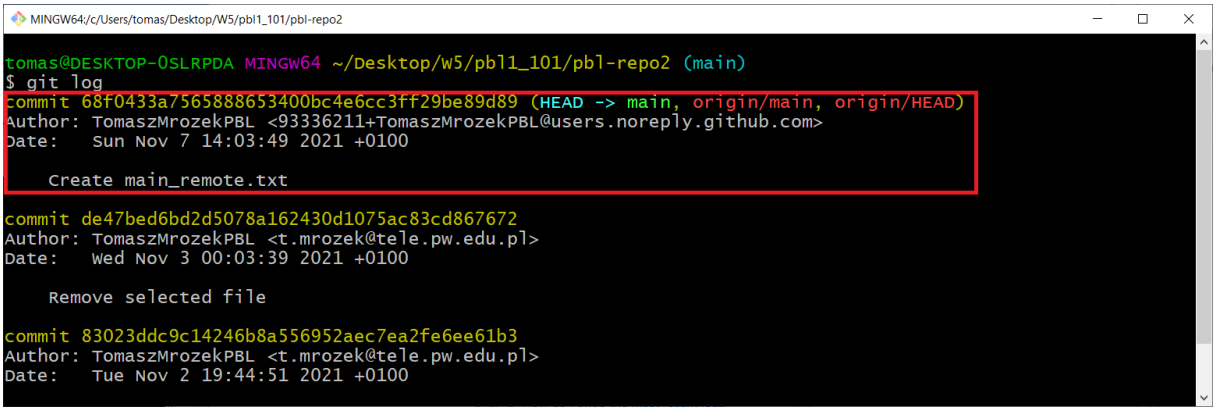
W informacji z konsoli widać, że wstawiono 1 plik „main_remote.txt | 1 +”.



Następnie należy przejść do folderu lokalnego repozytorium. Plik został dodany.



Wywołując komendę „git log” widać na samym początku informację z commita zdalnego (czerwona ramka):



Jeśli w naszym repozytorium dokonamy dużo zmiany i wybierzemy komendę „git log” to w oknie konsoli wszystkie „logi” nie będą się mieścić. Na końcu konsoli wyświetlony zostanie dwukropek (czerwona ramka w konsoli poniżej). Można śledzić wszystkie logi do samego końca naciskając enter. Każde kliknięcie odsłoni kolejny wiersz w konsoli.



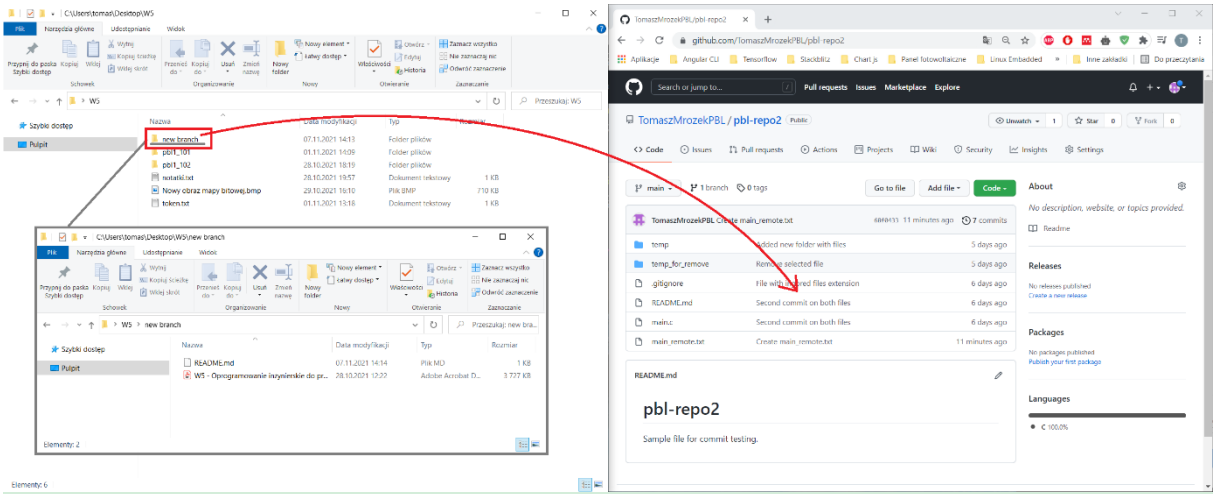
Gdy zostaną wyświetlone wszystkie logi na samym końcu konsoli zostanie wyświetlony komunikat (END).



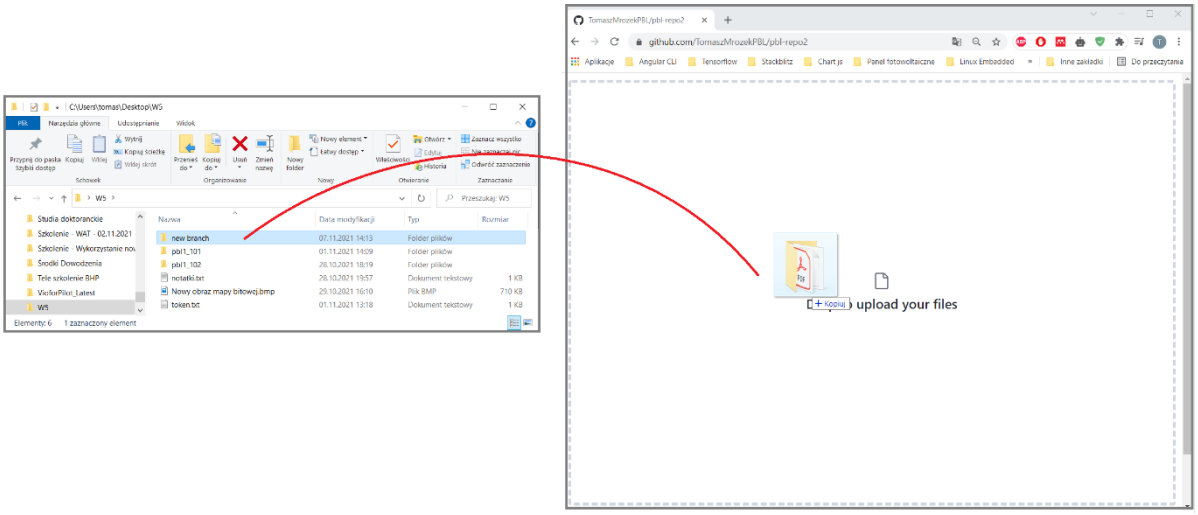
Aby wyjść z logów i móc ponownie wpisywać komendy w konsoli należy wybrać klawisz „Q” z klawiatury.



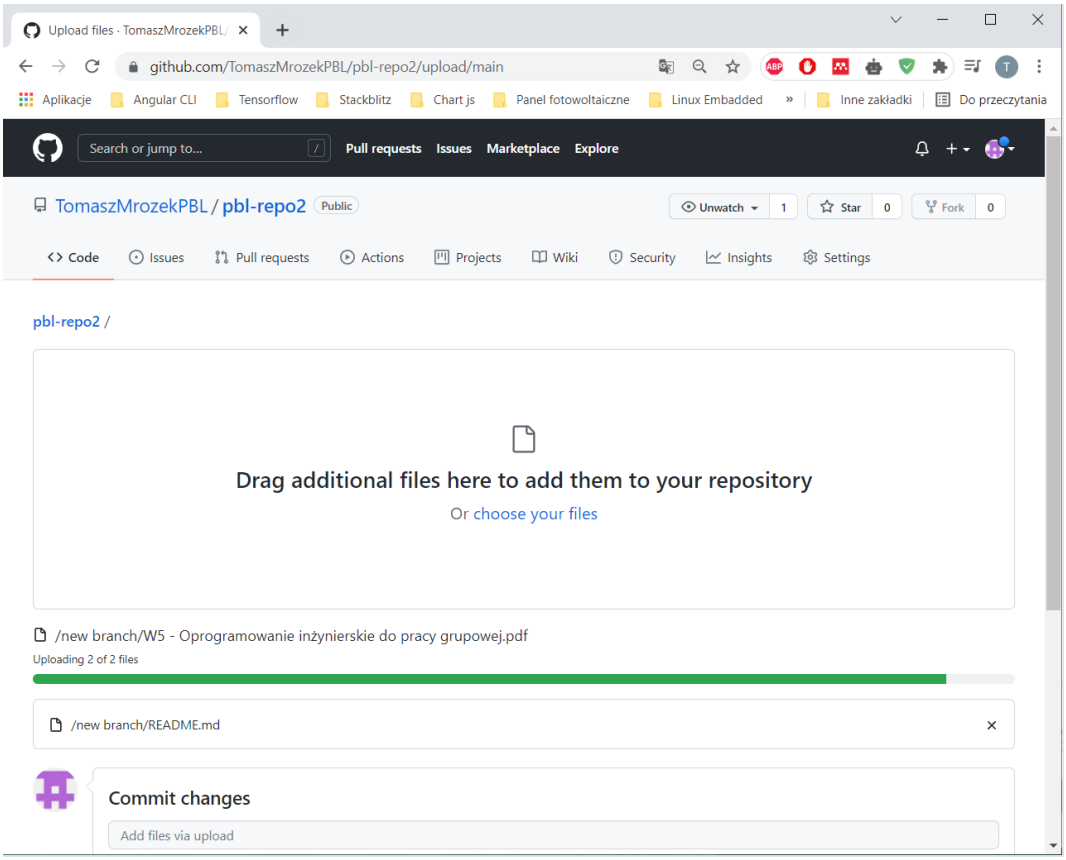
Powyższy przykład dotyczył dodania pliku. Co, jeśli chcemy dodać folder z plikami? Aby to zrobić należy w dowolnej lokalizacji na komputerze stworzyć nowy folder, który będziemy chcieli przenieść na repozytorium zdalne. W folderze tym tworzymy (lud dodajemy) również przykładowe pliki. Następnie przeciągamy cały folder nad otwarte repozytorium zdalne jak pokazano na poniższym obrazku.



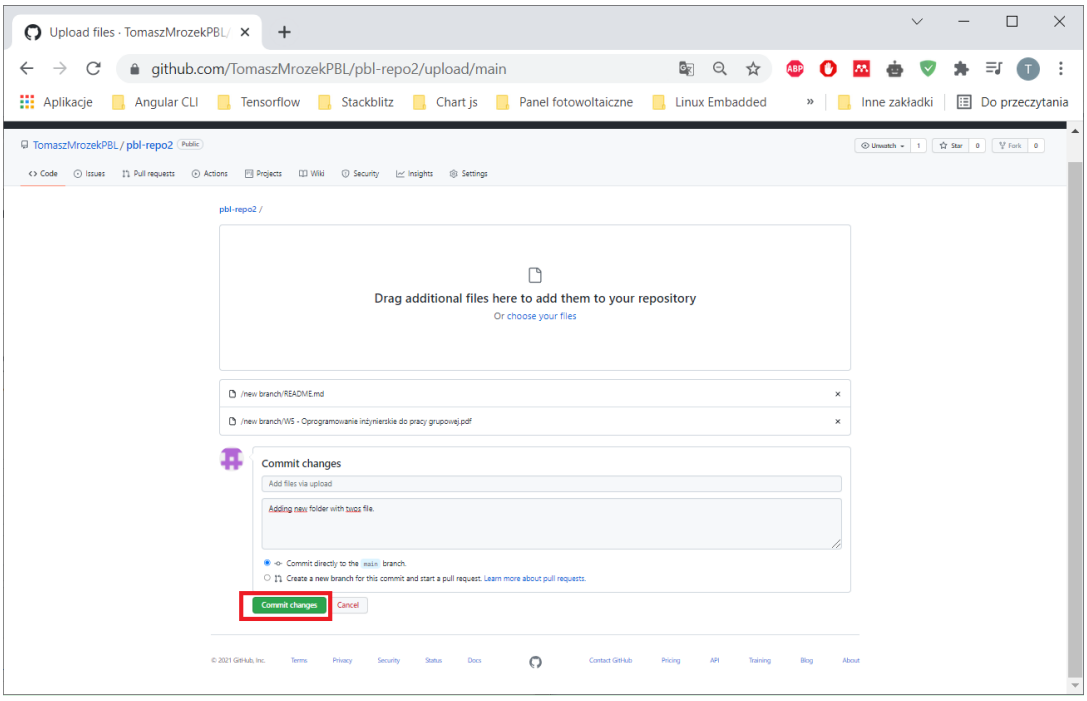
W momencie przeciągania, nad polem przeglądarki pokaże się informacja o możliwości przesłania plików:



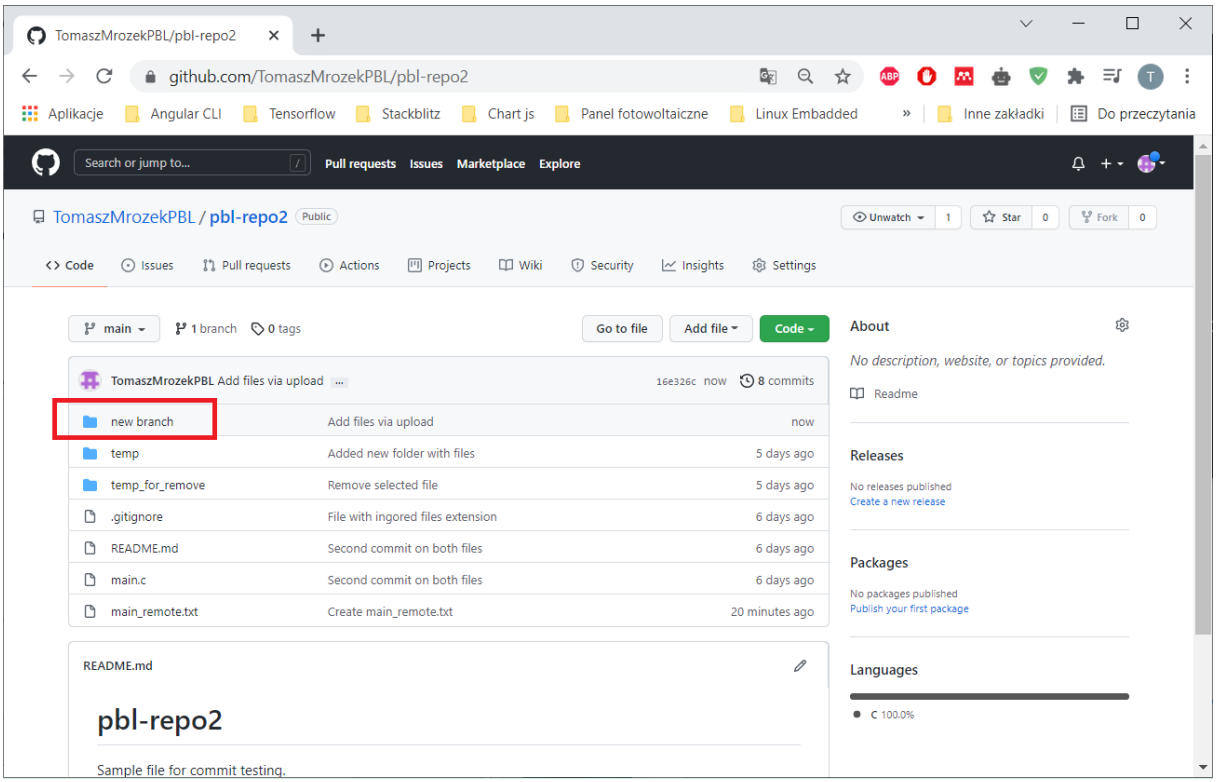
Po przeniesieniu folderu w polu przeglądarki wyświetli się zielony pasek postępu informujący o dodawanych plikach do repozytorium zdalnego.



Po dojściu do końca zielonego pasku należy przewinąć stronę do dołu i dodać informacje (komentarz) o commit, a następnie kliknąć w zielony button „Commit changes”.



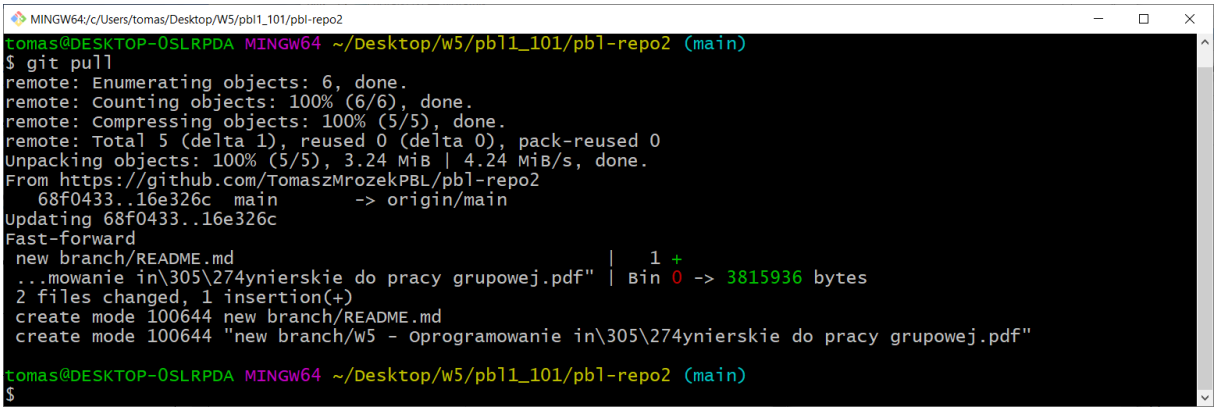
Nowy folder wraz z plikami został dodany do repozytorium.



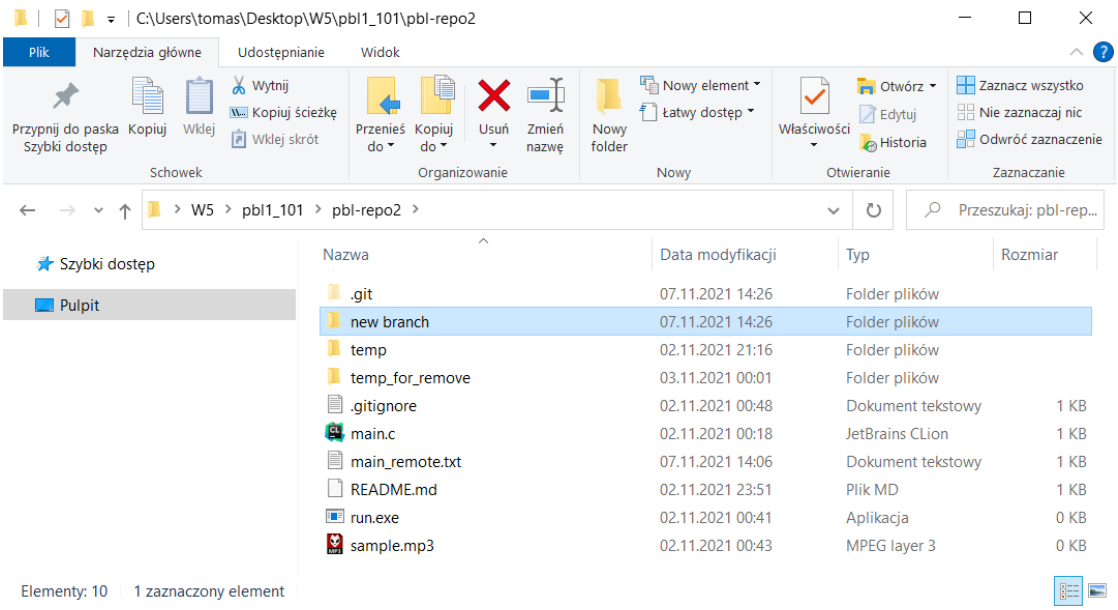
Zmiany są dostępne w repozytorium zdalnym. Należy teraz zaktualizować repozytorium lokalne. Aby to zrobić należy otworzyć konsole w folderze repozytorium i wpisać następującą komendę:

```
git pull
```

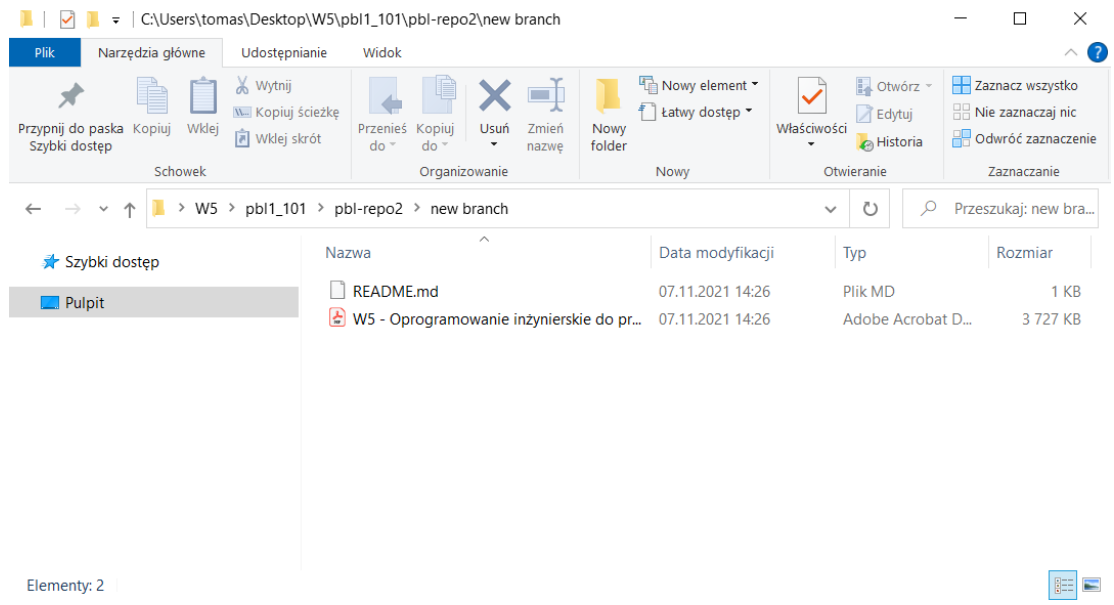
Jeśli wszystko zostało wykonane poprawnie w konsoli wyświetlą się informacje o dodanych plikach i folderze.



W celu sprawdzenia należy przejść do folderu z repozytorium i sprawdzić, czy został dodany nowy folder wraz z plikami:



Do repozytorium został poprawnie dodany folder „new branch” wraz z plikami:



Można również sprawdzić polecenie „git log” i zaobserwować dodaną informacje o commit.



Usuwanie nowo dodanych plików/folderów należy zrobić zgodnie z poznaną wcześniej procedurą. Wpisując komendę „ls” możemy zobaczyć dostępne w folderze repozytorium pliki i foldery. Następnie należy samodzielnie usunąć plik „main_remote.txt” (zakładamy, że usuniemy ten właśnie plik) z repozytorium lokalnego.

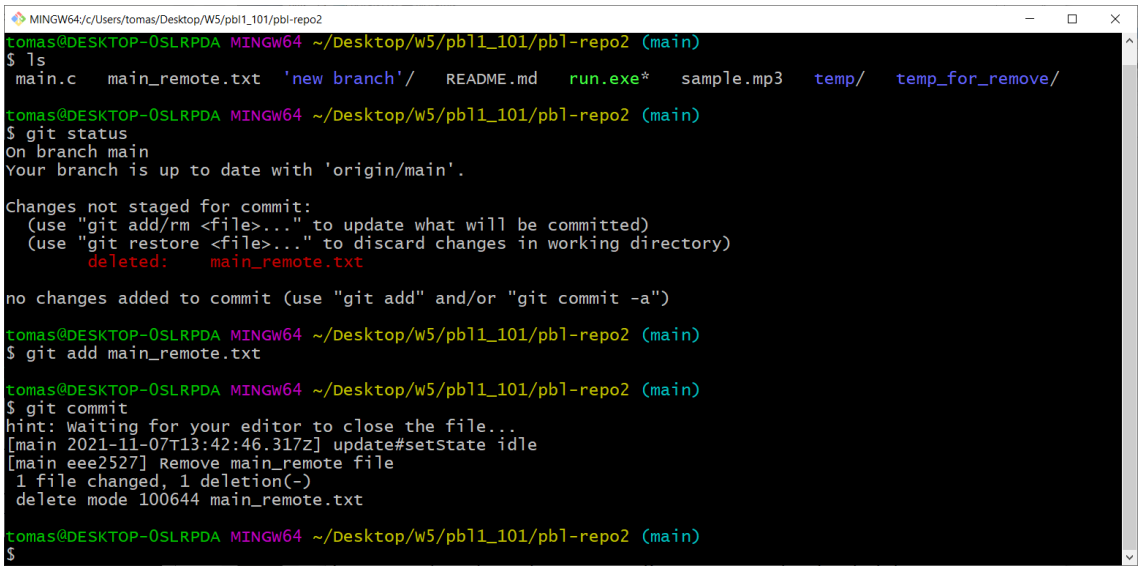
Następnie należy w konsoli wpisać komendę:

```
git add main_remote.txt
```

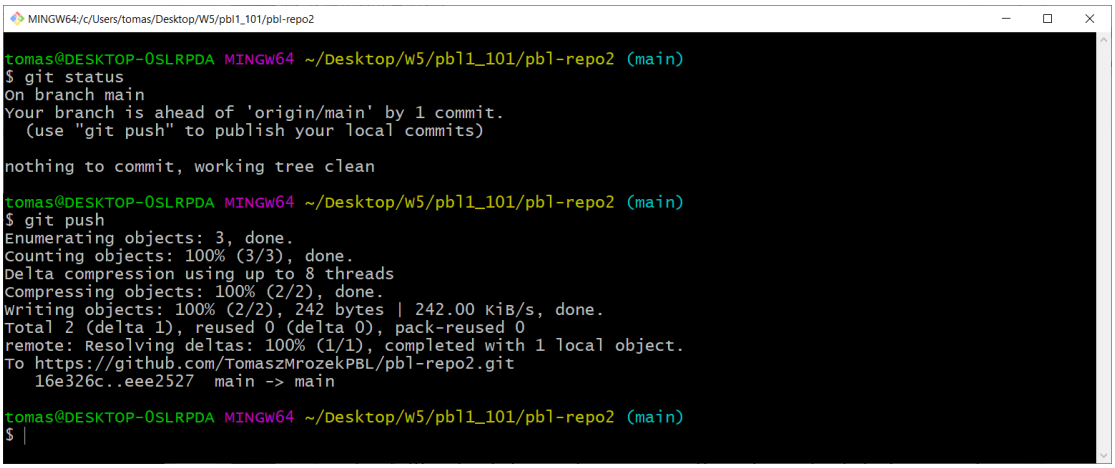
oraz

```
git commit
```

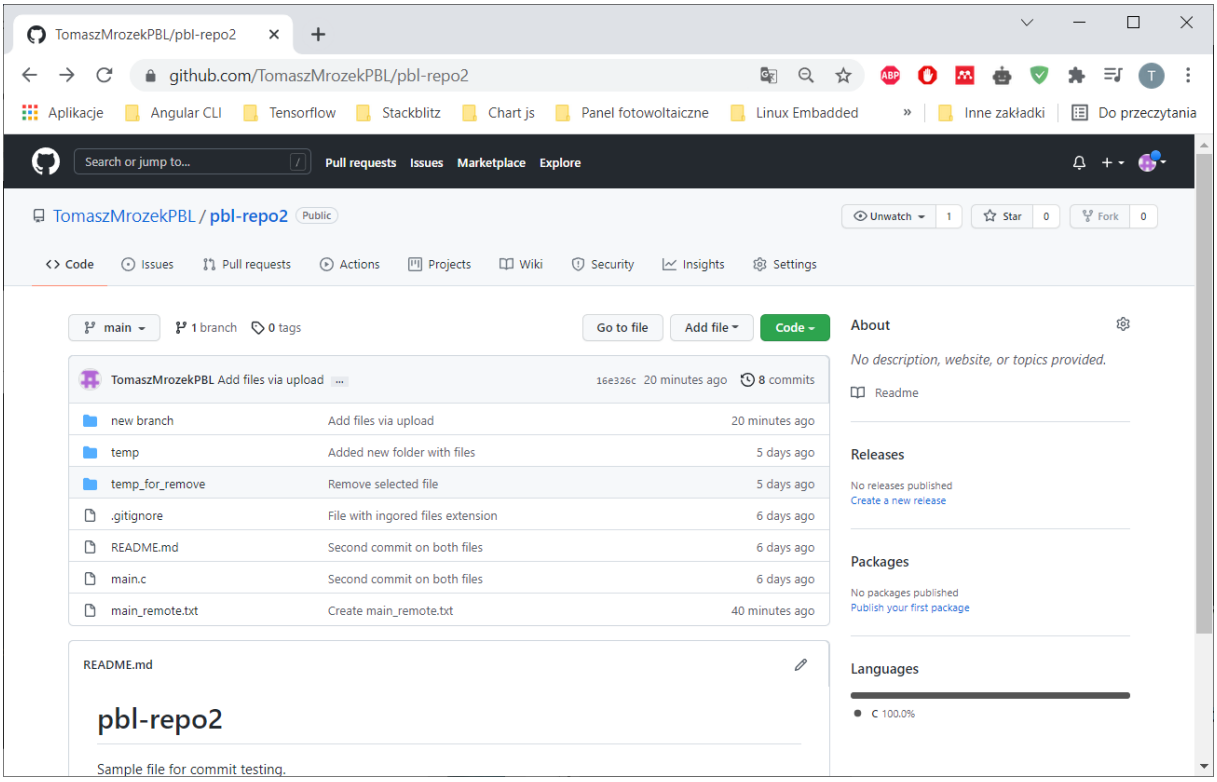
Jeśli po komendzie „commit” nie wpiszemy komentarza to automatycznie zostanie wyświetlony preferowany edytor tekstu, w którym należy wpisać informację do commita.



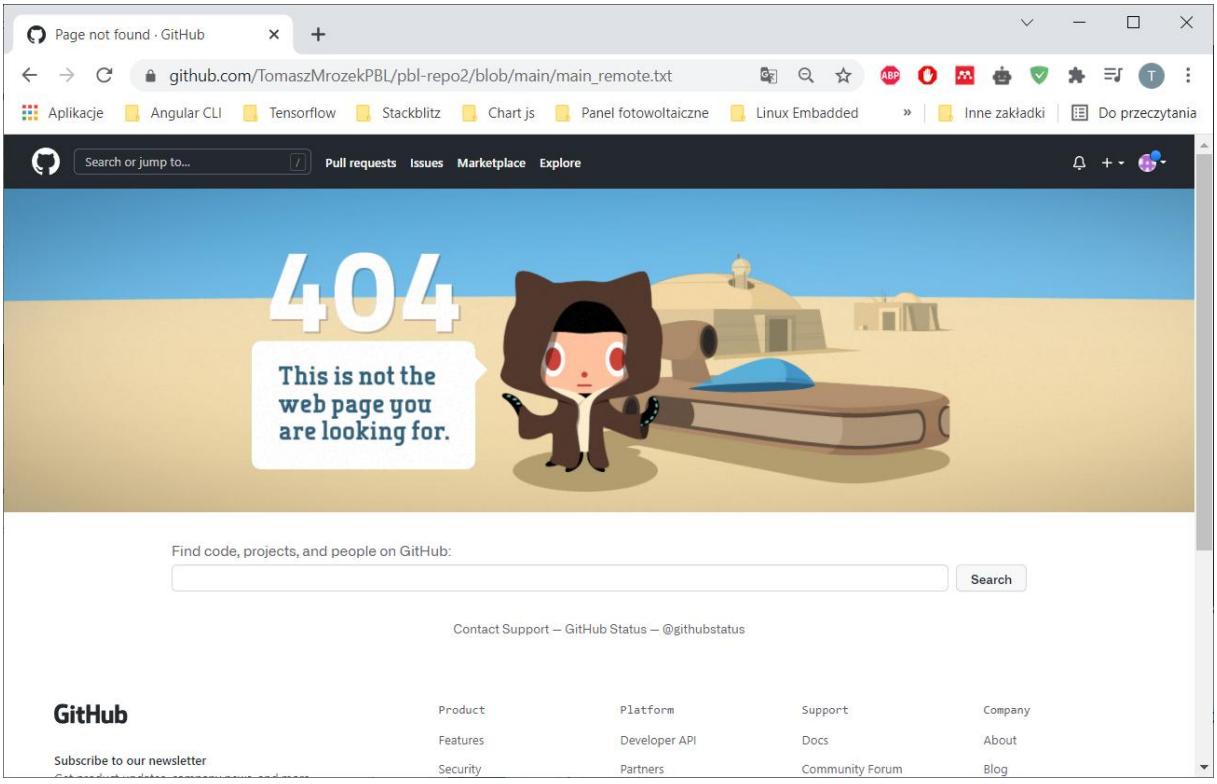
Następnie po wyświetleniu informacji o usunięciu pliku, można wybrać komendę „git status”, gdzie zostaniemy poinformowani o możliwości synchronizacji repozytorium zdalnego z lokalnym. Należy wpisać komendę „git push”.



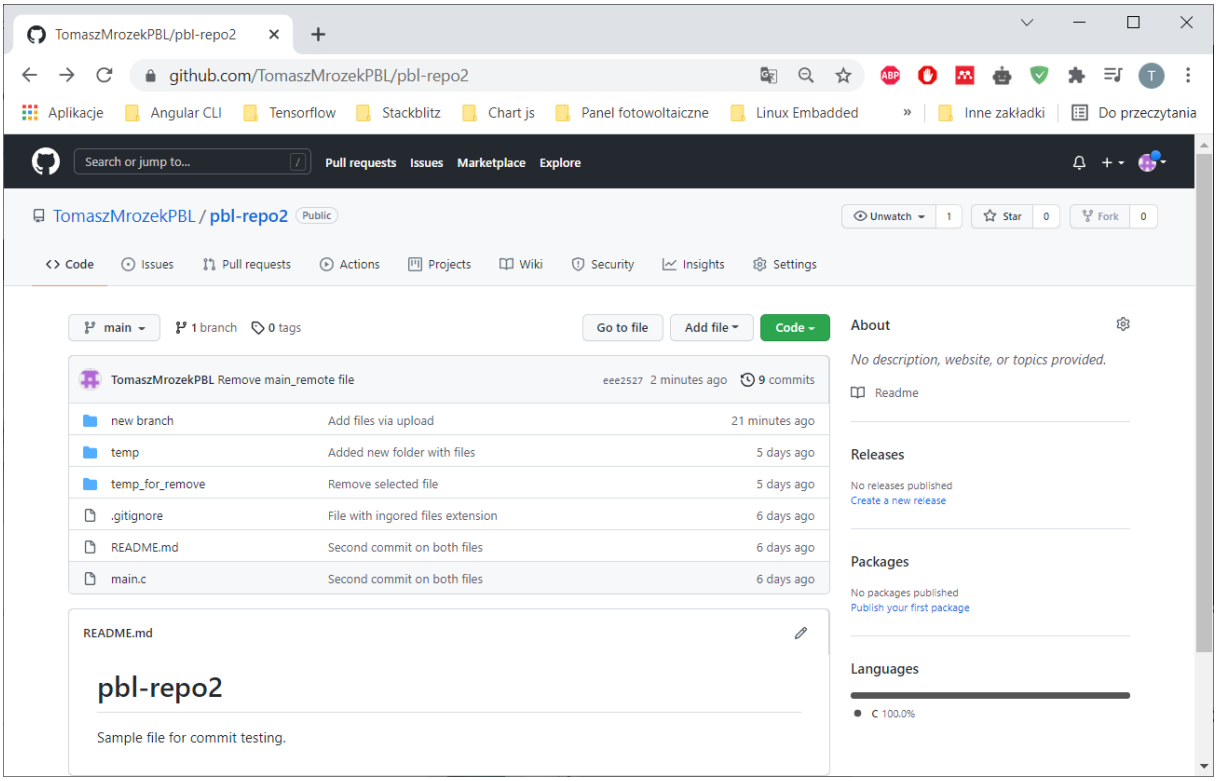
Po ukończeniu procesu „pushowania”, należy przejść do repozytorium zdalnego i odświeżyć stronę. Plik „main_remote.txt” został usunięty.



Jeśli nie zdążymy odświeżyć strony i klikniemy w plik „main_remote.txt” to zostaniemy przeniesieni na nieistniejącą stronę, ponieważ dany plik został już usunięty.

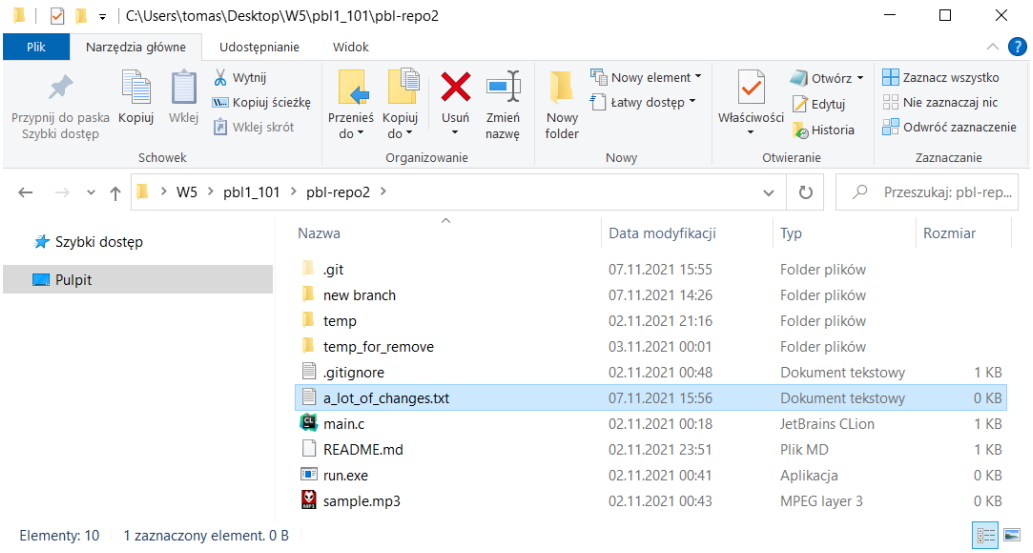


Podany plik został usunięty i nie ma do niego dostępu. Jeśli widać go jeszcze na liście plików należy odświeżyć stronę (klawisz F5).



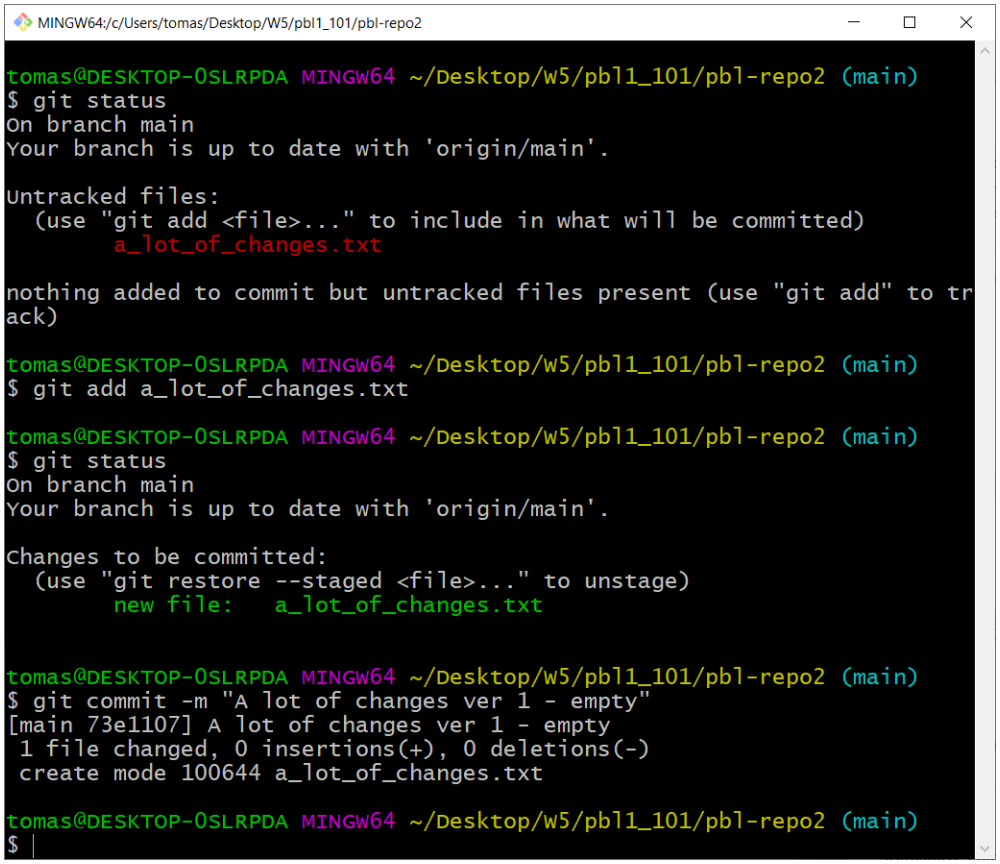
j) Przywracanie plików w wersji historycznej

Skupmy się na przykładzie przywracania plików do wersji historycznej. Stwórzmy w tym celu plik w repozytorium lokalnym, na którym dokonamy kilku zmian. Stwórzmy np. pusty plik tekstowy „a_lot_of_changes.txt”.



Uruchamiamy konsolę i dodajemy commit, wpisując kolejno komendy:

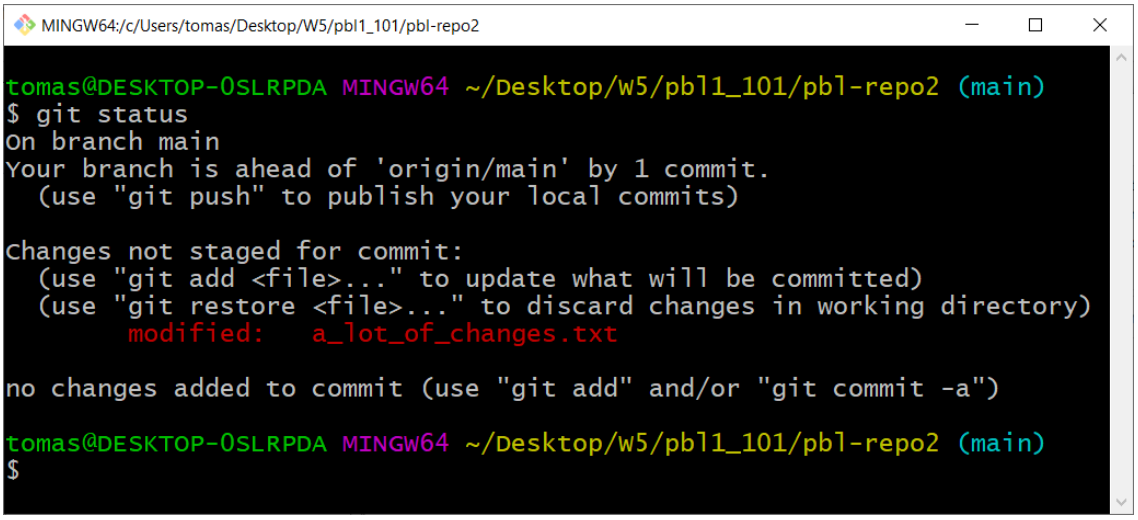
```
git add a_lot_of_changes.txt
git commit -m "A lot of changes ver 1 - empty"
```



Edytujmy plik tekstowy i dodajmy do niego wiersz:

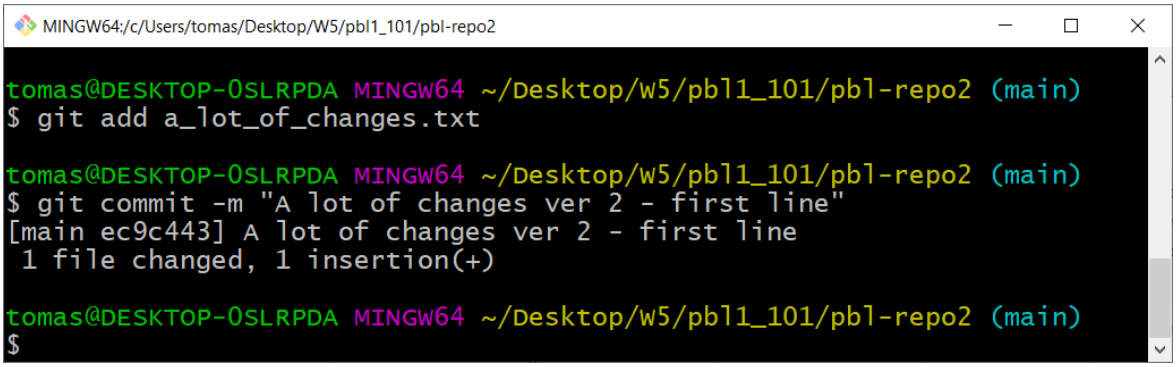
```
Wersja 2 – Druga zmiana – wersja 07.11.2021
```

Sprawdźmy status (git status):



Dodajmy commita ze zmianami:

```
git add a_lot_of_changes.txt
git commit -m "A lot of changes ver 2 – first line"
```



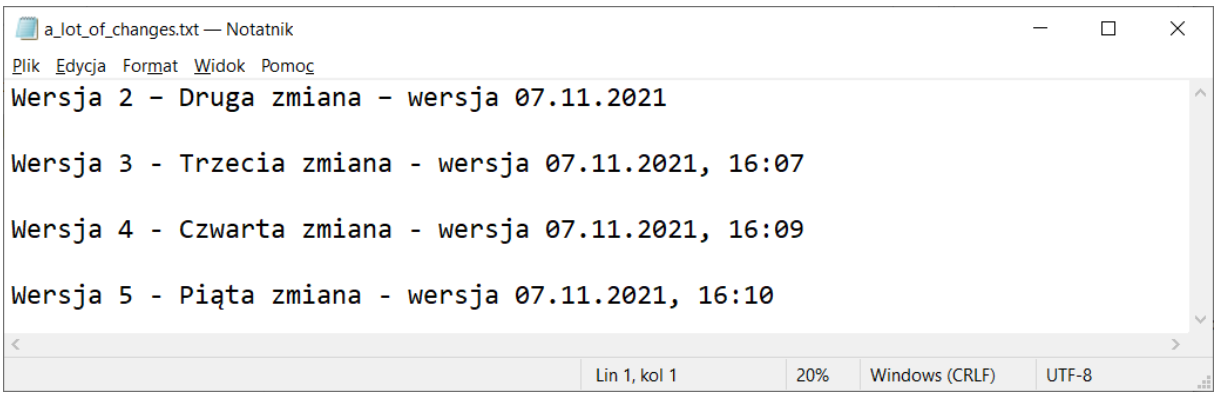
Powtórzmy zmiany w pliku trzykrotnie tzn. dodajmy jeszcze trzy nowe wiersze (po każdym wierszu należy dodać commita (tzn. git add … + git commit …).

Wersja 3 - Trzecia zmiana - wersja 07.11.2021, 16:07
git add a_lot_of_changes.txt git commit -m "A lot of changes ver 3 – second line"

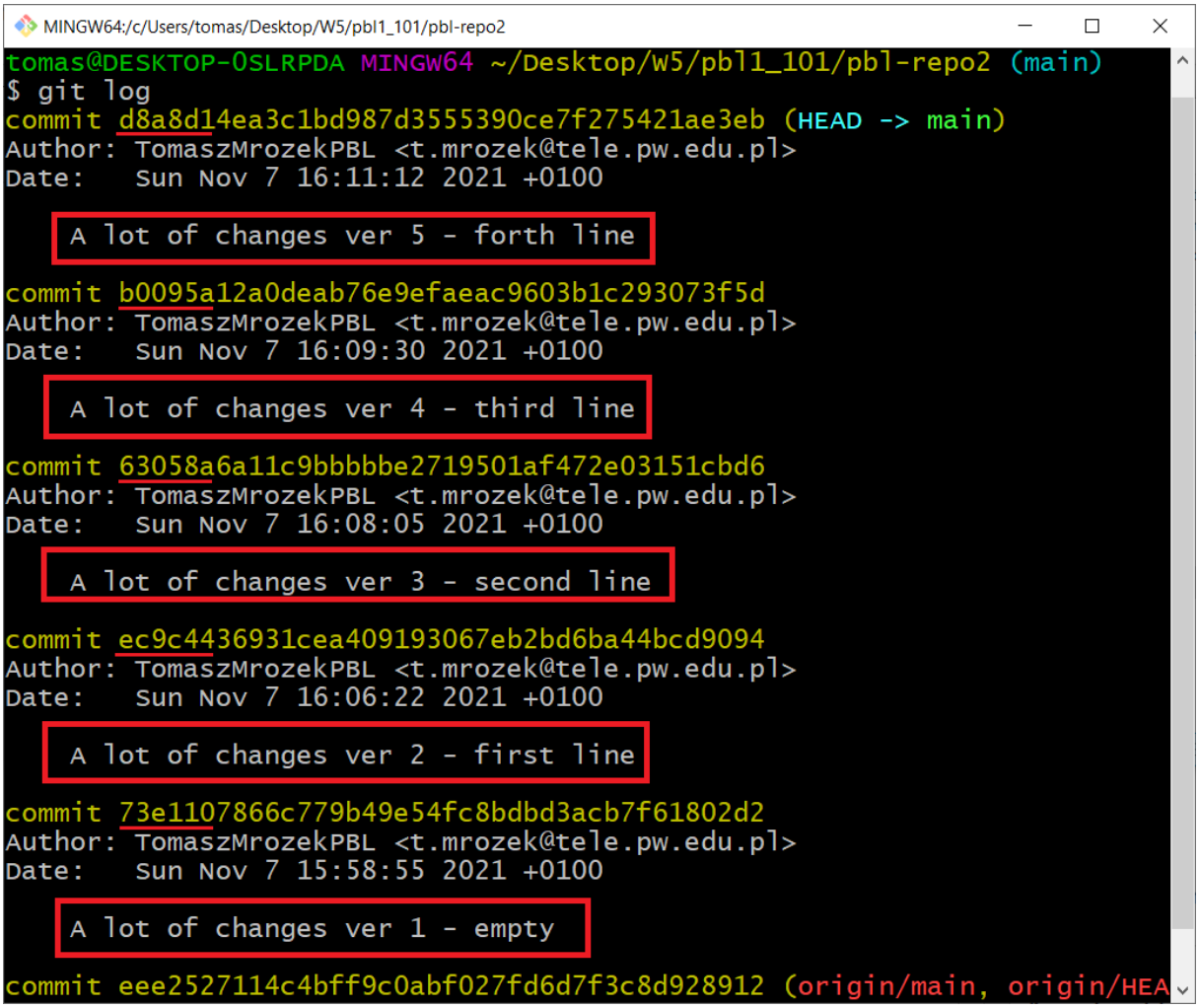
Wersja 4 - Czwarta zmiana - wersja 07.11.2021, 16:09
git add a_lot_of_changes.txt git commit -m "A lot of changes ver 4 – third line"

Wersja 5 - Piąta zmiana - wersja 07.11.2021, 16:10
git add a_lot_of_changes.txt git commit -m "A lot of changes ver 5 – forth line"

Zawartość pliku tekstowego powinna wyglądać następująco:

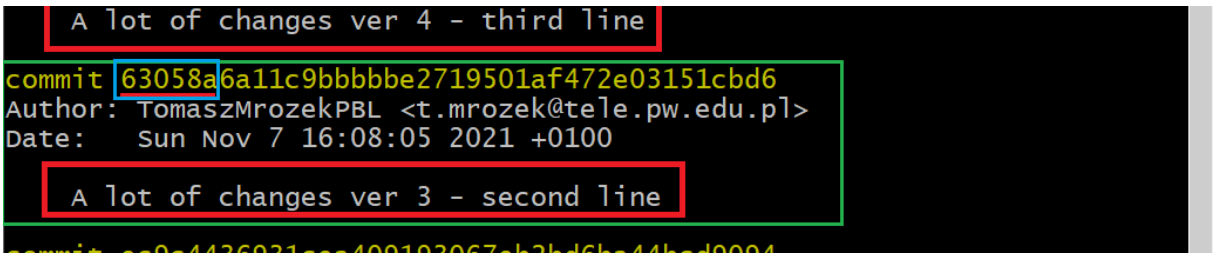


Następnie sprawdźmy logi (git log):



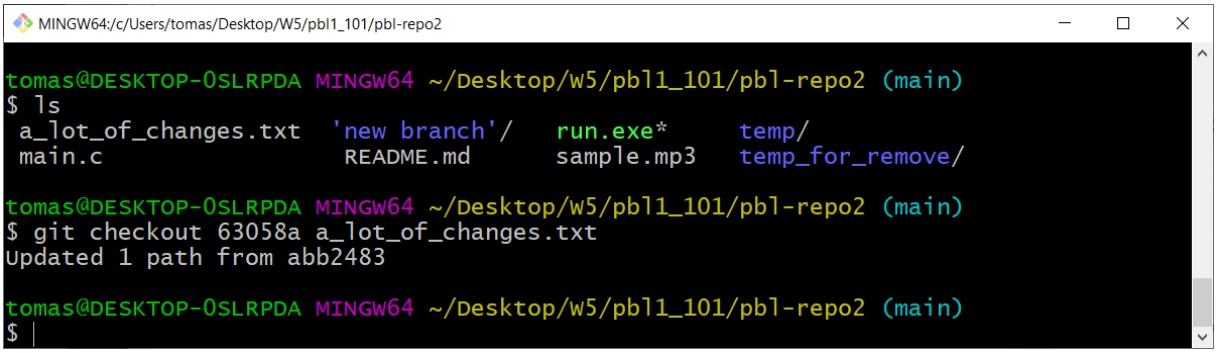
W ramkach zaznaczono komentarze commitów natomiast podkreślnikiem początkowe fragmenty kodu (SHA) do każdego commita.

Załóżmy, że chcemy przywrócić zawartość pliku „a_lot_of_changes.txt” do trzeciej zmiany (drugiego wierszu w pliku, pierwsza zmiana to utworzenie pustego pliku). Sprawdźmy początek kodu „SHA” dla pożądanej zmiany:

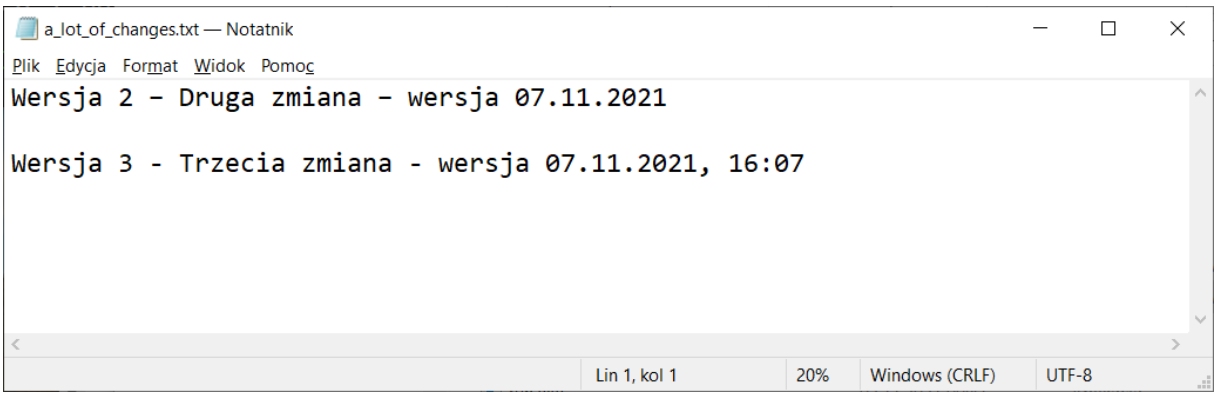


Aby przywrócić zawartość pliku do pożądanego stanu należy wpisać w konsoli:

```
git checkout 63058a a_lof_of_changes.txt
```

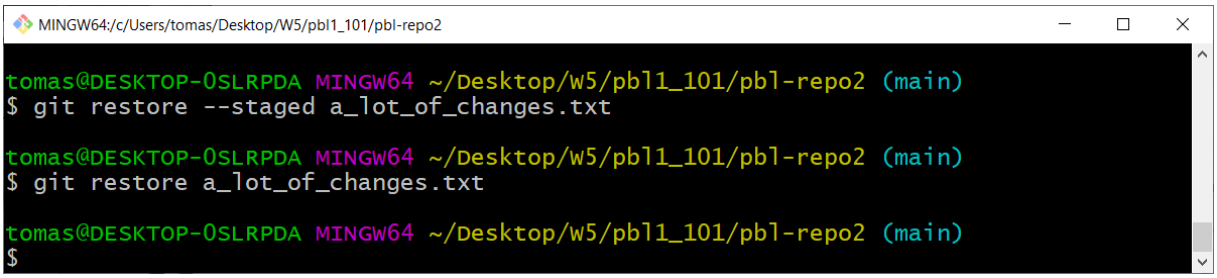


Sprawdźmy zawartość pliku „a_lot_of_changes.txt”:

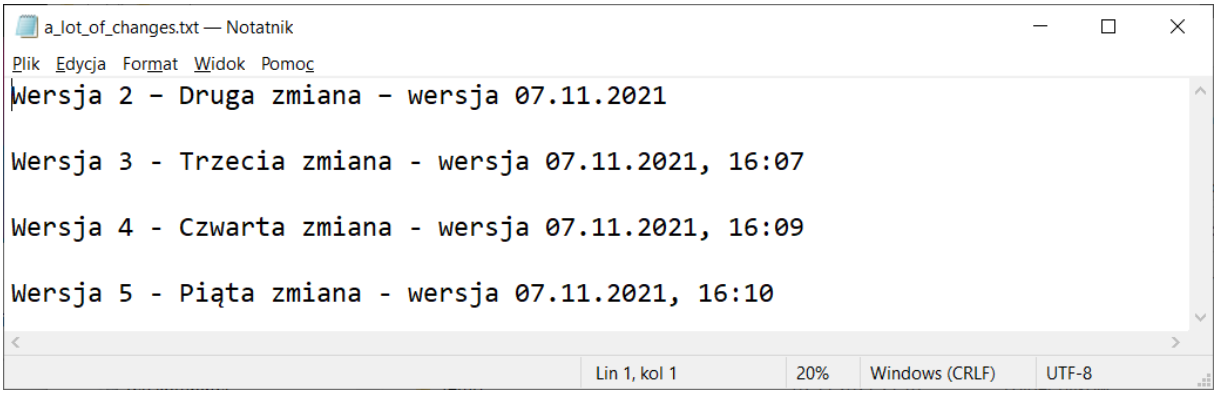


Treść pliku została przywrócona do drugiego komentarza. Aby przywrócić plik do najnowszej wersji należy wpisać komendy:

```
git restore --staged a_lot_of_changes.txt
git restore a_lot_of_changes.txt
```

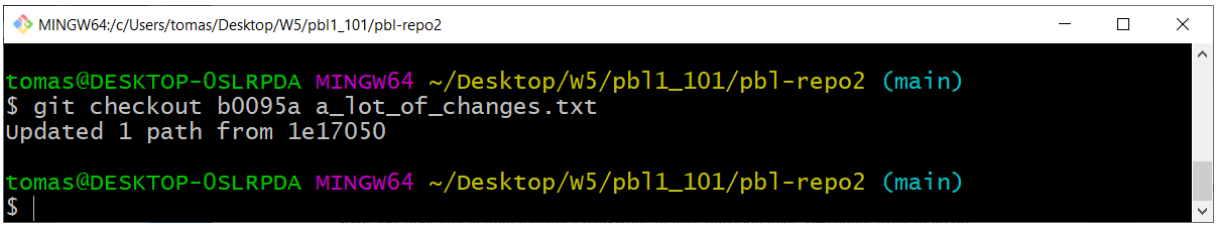


Zawartość pliku wygląda następująco:

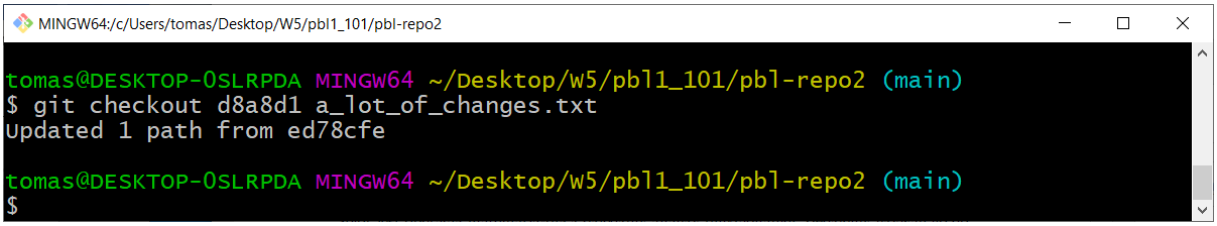


Pomiędzy wersjami pliku można się przełączać z wykorzystaniem wcześniejszej komendy „checkout" tzn. z poprzedniego stanu, dwóch wierszy w pliku „a_lot_of_changes.txt" do dowolnego stanu np. wersji 4 lub wersji 5 wpisując odpowiednie SHA:

- do wersji 4:



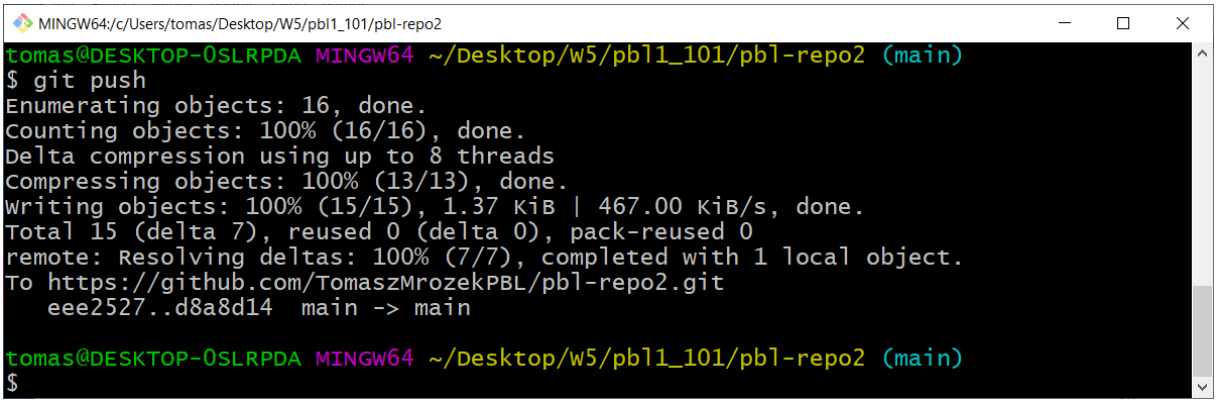
- do wersji 5:



gdzie ostatecznie:

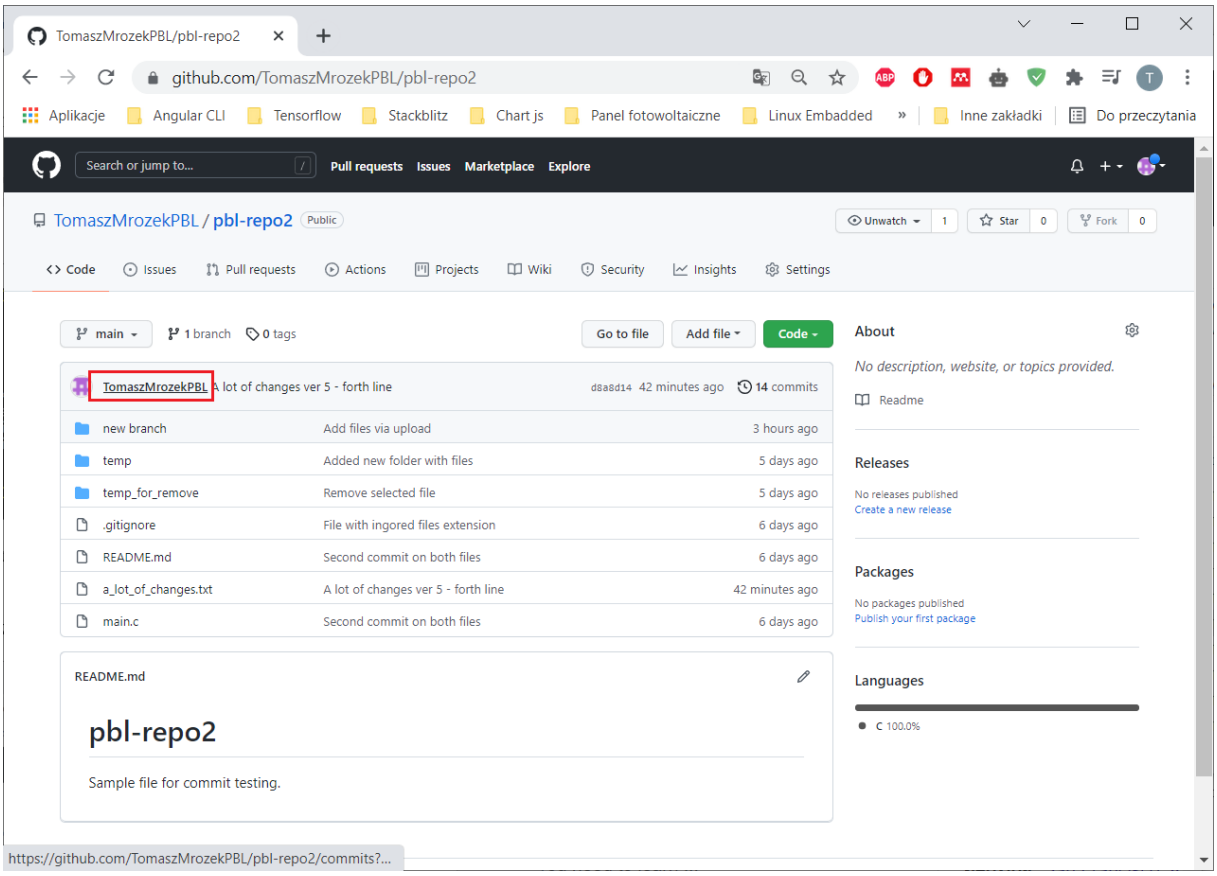


W ramach treningu sprawdź, dla jakiego początku kodu „SHA" (jakiej długości kodu, tutaj 6) można się przełączać między wersjami pliku. Na tym etapie można zsynchronizować repozytorium zdalne z lokalnym z wykorzystaniem komendy „git push".

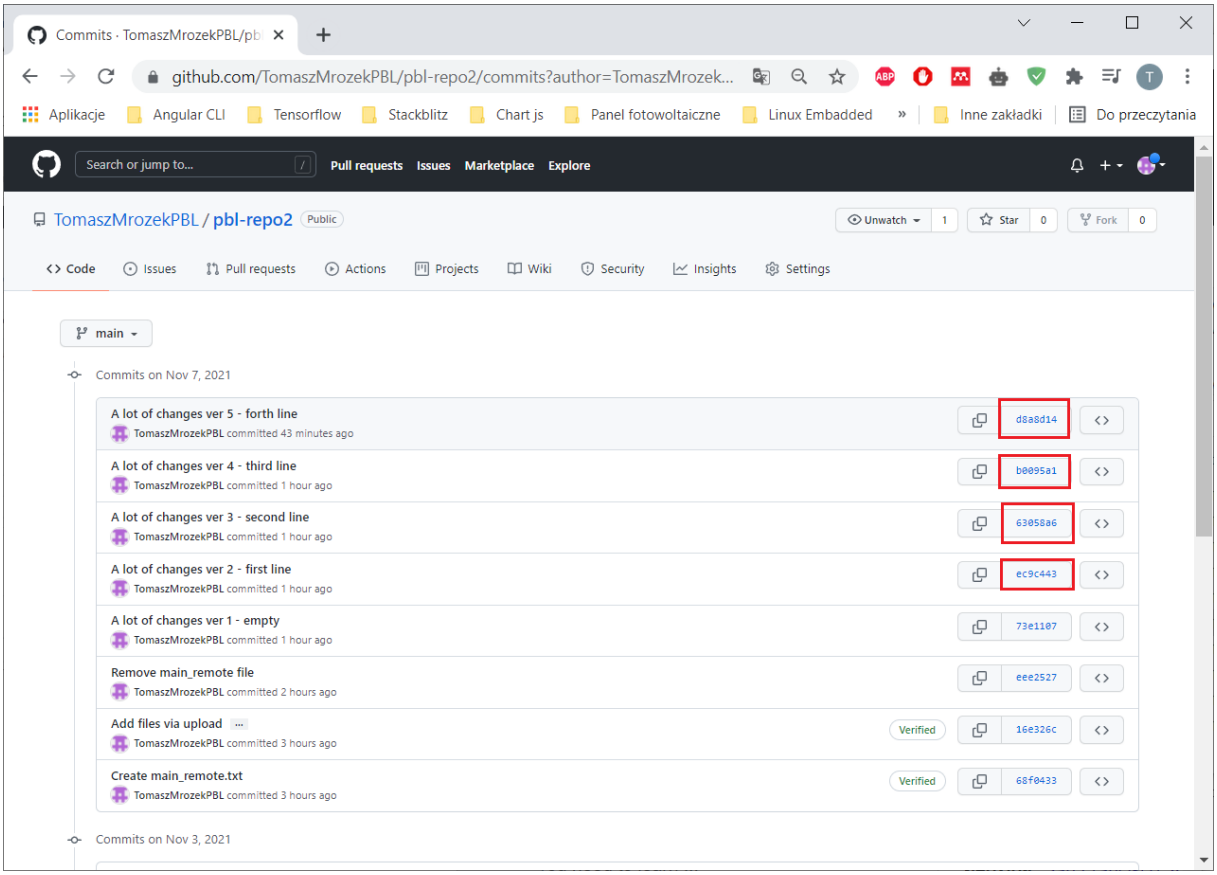


Aby zobaczyć zmiany w repozytorium zdalnym należy odświeżyć stronę GitHub lub zalogować się na konto.

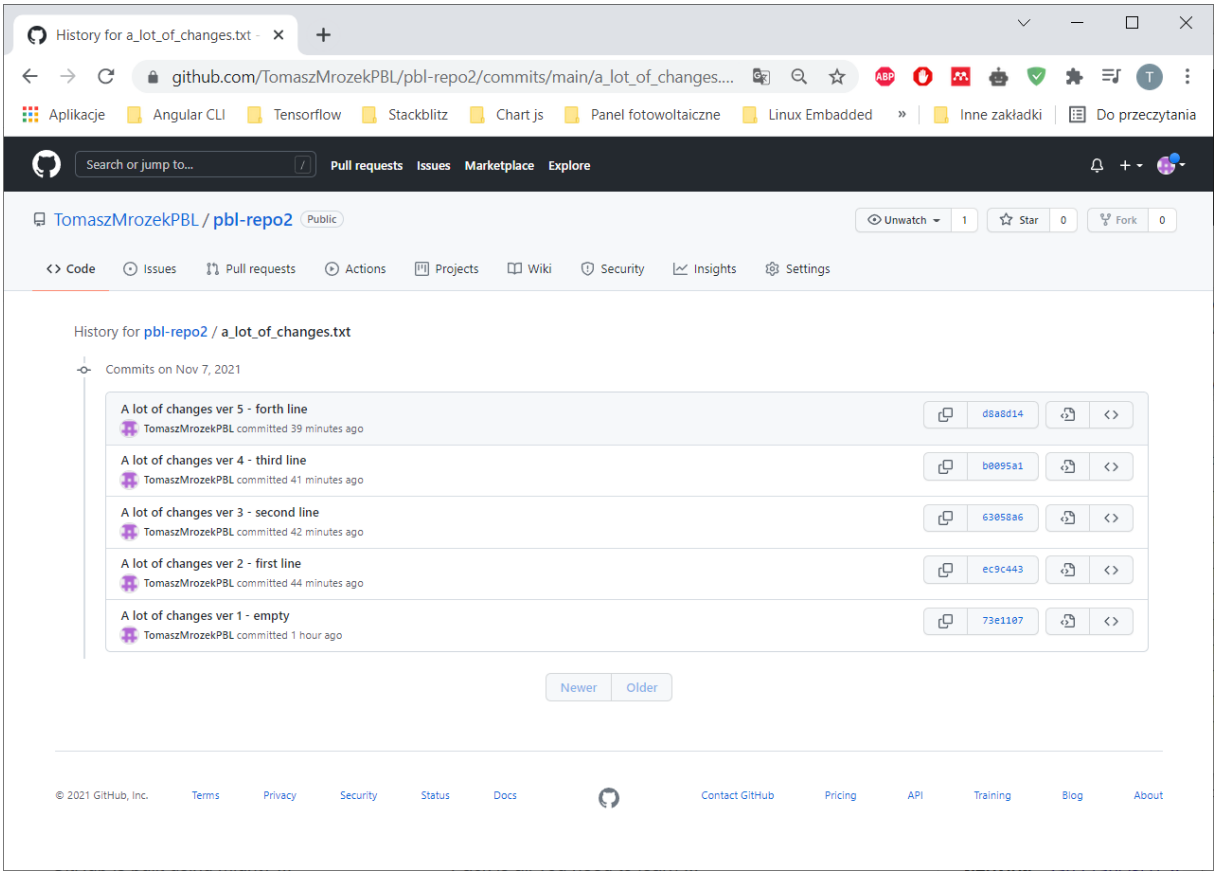
Warto również przenalizować repozytorium z pozycji zdalnej przez GitHuba. Klikając w czerwoną ramkę (nazwę użytkownika),



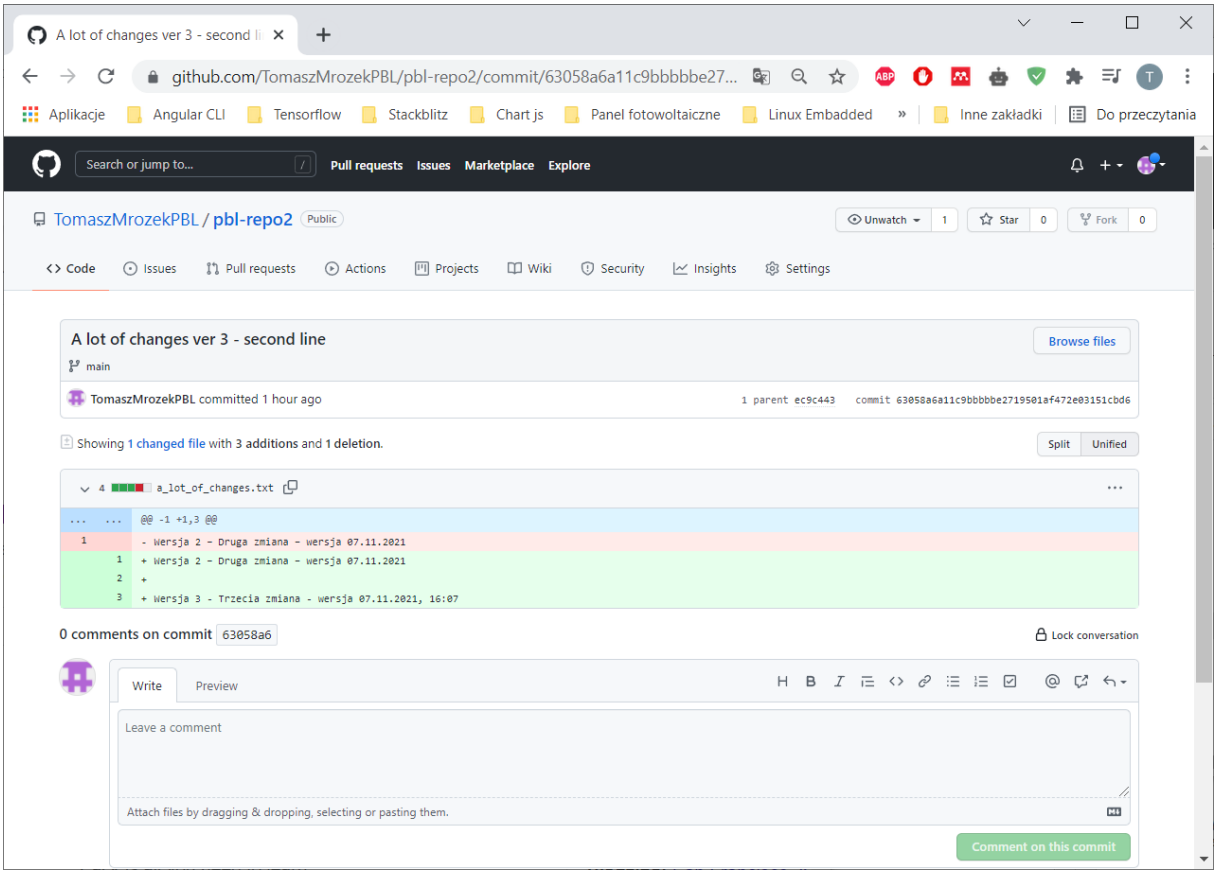
można uzyskać dostęp do wszystkich commitów z wyszczególnionymi numerami SHA:



Takie same informacje można uzyskać po wybraniu danego pliku:



Klikając poszczególny commit można uzyskać podgląd na zawartość pliku wraz z rożnymi kolorami modyfikowanej zawartości pliku np.



k) Uwierzytelnienie za pomocą kluczy publicznych SSH

Istnieje możliwość dostępu do repozytorium za pomocą klucza SSH. W ramach ćwiczenia zapoznaj się ze sposobem generowania takiego klucza. Wszystkie niezbędne informacje uzyskasz pod linkiem:

https://docs.github.com/en/authentication/connecting-to-github-with-ssh